Fake Multimedia Content Detection Using Deep Learning Technique

Developed By:

Abdullah Mohammad Hashem Tayeh
Noor Zeyad Saleem Barahmeh
Dana Omar Saleh Abdulqader
Saleh Abdeljabbar Subhi Mohammad

Supervised By:
Dr. Rasha Al-Bashaireh

Computer Science Department
Faculty of Information & Communications Technology
Tafila Technical University
Fall 2025-2026

ACKNOWLEDGMENTS

We would like to extend our sincere thanks for the help of all those people whose contributions made this project a success.

First and foremost, we would like to extend our sincere gratitude to our supervisor, Dr. Rasha Al-Bashaireh, for her incredible support and contributions to this study. Without her guidance and support, it would not have been possible to develop this study and increase the quality of the output that we have achieved in this research.

We further appreciate Tafila Technical University for granting us an ideal academic atmosphere and facilitating access to the required tools for completing this project.

In addition to this, we would like to extend our heartfelt gratitude to our family members for their undying support, patience, and encouragement in all of us while pursuing our academics.

Finally, we would like to extend our thanks to our friends for the support, assistance, and encouragement necessary for surmounting challenges in completing the project tasks.

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION

1. INTRODUCTION

1.1 Problem Identification and Motivation:

Artificial Intelligence (AI) is widely recognized as an effective tool with both positive and negative implications. Due to its rapid development and widespread adoption, creating forged visual, written, or auditory content has become significantly easier. [1] [2]

One can only imagine how dangerous that reality could be, with fake content spreading everywhere. How often have we seen news, images, or voices that looked genuinely authentic at first? How many videos of public figures discussing something important have we seen, only to find out later that everything was created by AI and completely fake? [2]

Given the proliferation of AI-generated content and the tools that generate it, some risks have emerged. One of these risks is the publication of AI-generated content on social media platforms (which are designed to transmit information and news). This may lead to the transmission of wrong information. So, we need to detect if a page is publishing fake, unverified content. [1]

In the continually changing digital world, "DeepFakes" -a type of synthetic media created using artificial intelligence-have been recognized as an essential security risk. Although much has been said about "DeepFaked" videos, "DeepFaked" Audio is actually a more dangerous genre. [10] [2]

It is possible to create copies of human voices with chilling realism, which can be used to create very cunning "phone scams" as well as forge people's identities as part of spreading "disinformation." The problem that this project aims to solve is: The present level of human intelligence, as well as current software, is unable to distinguish between human speech versus its imitation by artificial intelligence. [2]

1.2 Problem Background:

The rise of AI-generated deepfakes, synthetic audio, and text is making it increasingly difficult to distinguish fact from fiction. This widespread digital deception threatens social trust, personal security, and the integrity of democratic processes. Within our project, we will collect and analyze audio, visual, and textual data, and train a model capable of detecting whether a page or account

is publishing fake content. This will be achieved by algorithms like CNN (Convolutional Neural Network), RNN (Recurrent Neural Networks), LSTM (Long Short-Term Memory). [3]

How many times have we read a report that seems to be written by a human but is the product of artificial intelligence? Indeed, not everything that circulates in the news is factual.[6]

Now, since social media platforms are everywhere and used by everyone, a solution to differentiate between fabricated and authentic content is highly needed. Here is where our role kicks in: to come up with software that can determine the authenticity of content, whether images, audio, or other formats, using Artificial Intelligence techniques. In principle, we are fighting AI with AI.[8]

1.3 Problem Related Work

To identify fake content on social media, we need to analyze different types of media, including video, text, images, and audio. We can train separate models for each type by using deep learning algorithms. This approach helps us achieve better accuracy.

1.3.1 Video Deepfake Detection

Early work on video deepfake detection mainly focused on classifying images frame by frame. Researchers initially used standard Convolutional Neural Networks (CNNs) like Xception to detect artifacts in individual frames. However, these spatial-only methods often had trouble with high-quality deepfakes, especially when the manipulations varied over time, such as unnatural blinking or lip movements. To address this challenge, benchmarks like FaceForensics++ were introduced to evaluate detection methods across various manipulation techniques like FaceSwap and DeepFakes. Modern methods now combine efficient feature extractors like EfficientNet with temporal sequence analyzers, such as Long Short-Term Memory (LSTM) networks. This combination allows models to remember context across frames and identify temporal anomalies that single-frame detectors might miss.[7][1][5][8]

1.3.2 AI Text Detection

With the rise of Large Language Models (LLMs), it has become much harder to distinguish machine-generated text from human writing. Traditional methods relied on statistical features like word frequency, but these often do not work well for more advanced AI outputs. Recent progress in Natural Language Processing (NLP) has moved toward neural methods. As noted in early studies on machine reading, teaching machines to understand context is essential.

Therefore, current detection systems use Bidirectional LSTMs (Bi-LSTM) and 1D Convolutions to capture local stylistic patterns and long-range semantic connections. This setup helps differentiate the perfect grammar of AI from the natural variability found in human writing.[3]

1.3.3 Image Forgery Detection

In the realm of static images, generative models can create hyper-realistic faces and scenes. However, these processes often leave tiny pixel-level artifacts or frequency distortions. Research shows that using face detection and alignment algorithms like MTCNN is a crucial preprocessing step. It ensures classifiers focus on the relevant facial regions. By standardizing inputs, lightweight CNN architectures can be trained to classify images as "Real" or "AI-Generated" by detecting these subtle inconsistencies. Standard datasets, such as those from Google and Jigsaw, provide the necessary foundation for training these detection models.[6]

1.3.4 Audio Synthetic Speech Detection

For audio, recent research has shifted from analyzing raw audio waveforms to treating audio analysis as an image classification problem. Tools like Librosa turn audio into Mel-Spectrograms, which are visual representations of sound. Researchers can then use powerful CNNs originally designed for image recognition. Our work builds on this foundation but introduces a lightweight architecture called MobileNetV2, which offers faster inference compared to the heavier models used in earlier studies. [4]

To make these models more accessible and user-friendly, we designed a modern, efficient web app interface that also functions well on mobile devices. The connection between models and web pages is managed using Flask, which is both simple and effective.

CHAPTER 2. DATA

2. DATA

2.1 Datasets Description

This project will use different datasets to train the individual components of the system: one for detecting deep-fake videos and the other for detecting artificially generated text using AI.[3]

2.1.1 Video Dataset Description

To ensure robustness on our deepfake detection model, a hybrid dataset is created by incorporating two main sources: FaceForensics++ (FF++), and the Deep Fake Detection (DFD) dataset.[1]

- **FaceForensics++**

The ground truths of the training set are based on the FaceForensics++ benchmark. This set comprises about 192,000 pre-extracted image frames categorized into binary classes based on their intent and application:[2]

- Real: Pure frames taken from original videos, which were downloaded from YouTube. [2]
- Fake: Videos framed using four current automated methods for video falsification, namely DeepFakes, Face2Face, FaceSwap, and Face Deep Fake Detection (DFD) [1]
- Original Dataset: For complementing the data from the FF++ dataset, we used the Deep Fake Detection dataset, which consists of thousands of raw videos of actors under different lighting conditions. [2]

- **The Hybrid Unified Dataset:**

The last dataset ready for training is a combination of FF++ frames and faces extracted using the custom DFD approach. This technique avoids any potential bias arising from a dataset's unique encoding.

2.1.2 Text Dataset Description

The Text Classification component relies on a specially crafted multi-source dataset that can be used to adequately classify content as either human-written journalism or AI-written content. This multi-source dataset consists of three different classes. These classes are labeled as follows: Human Written Content, AI Generated Content ChatGPT, and AI Generated Content Gemini.[3]

- Human Written Corpus.[3]

Source: CNN/DailyMail dataset from Kaggle.[3]

Content: news articles written by news professionals that establish a benchmark for difficult, sensible, and fact-based human prose.

- AI Generated Corpus (Custom Scraped).[3]

A purpose-built Python scraper was developed to fetch responses from Large Language Models in a relevant and diverse manner.

Target Models: OpenAI's ChatGPT and Google's.

Generation Method: A graphical interface tool developed using tkinter and selenium (with the use of undetected_chromedriver) made it possible to automate browser actions in order to overcome common bot detectors. .

Strategy for Prompting: A range of engaging topics was used to input the LLMs through multi-threading to enable optimal data gathering.

2.1.3 Audio Dataset Description

We chose the "Release in the Wild" dataset because it's a tough set of audio recordings that was designed specifically for the purpose of deep-fake detection. [10]

- Structure: The data is divided into train, validation, and test directories. [10]
- Classes: It has two separate classes: [10]
- real: Genuine human speech recordings. [10]
- fake: Speech synthesized by different AI-based text-to-speech or voice conversion models. [10]
- Format: The original data will be audio files (.wav, .mp3) that are of varying lengths. [10]

2.1.3 Image Dataset Description

- This dataset contains images of 192 different scene categories, with both AI-generated and real-world images for each class. It is designed for research and benchmarking in computer vision, deep learning, and AI-generated image detection.[1]
- Key Features:

1. 192 Scene Classes: Includes diverse environments like forests, cities, beaches, deserts, and more.[2]
2. AI-Generated vs. Real Images: Each class contains images generated by AI models as well as real-world photographs.[9]
3. High-Quality Images: The dataset ensures a variety of resolutions and sources to improve model generalization.[3]

- Potential Use Cases:

1. AI-generated vs. real image classification [10]
2. Scene recognition and segmentation [10]
3. Training deep learning models for synthetic image detection [10]
4. Analyzing AI image generation trends [10]

2.2 Dataset Preprocessing Steps

1. Data preprocessing was performed in separate pipelines for video, text, audio and image data [10][2]

2.2.1 Video Preprocessing Steps

1. **Offline Facial Extraction** [6]

Face Detection: MTCNN is applied to detect faces from the video frames.

Strided Sampling: Frames were extracted with a stride value of 5 to 10 to remove redundancy.

Cropping: The detected faces are then cropped and saved as separate image files.

2. **Sequence Formation and Augmentation** [6]

- Sequence Generation: The frames were arranged into sequences of size 20 based on the temporal ordering.
- Class Balancing: WeightedRandomSampler was used to correct the imbalance in "Fake" and "Real" data.
- Image Augmentation: Input images were resized into 224 * 224 patches, normalized based on the ImageNet distribution, and then underwent color jitter.

You can see video data preprocessing pipeline here:

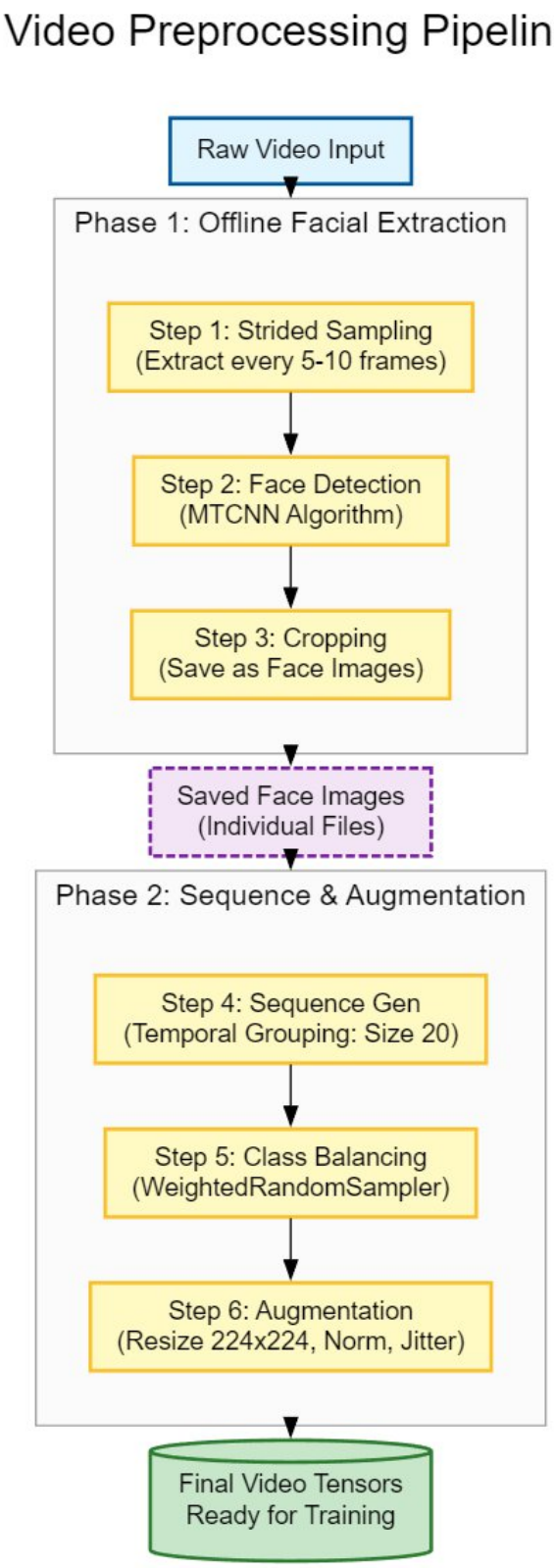

Figure1: Video dataset preprocessing pipeline

2.2.2 Text Preprocessing Steps

The raw text data went through a transformation pipeline: [3]

1. Tokenization: Made use of the TensorFlow Keras Tokenizer to turn raw text into sequences of integers according to word frequency. [3]
2. Sequence Padding: Integer value sequences were processed using the pad_sequences function from the TensorFlow library to have them of a standard length of MAX_LEN = 150. [3]
3. Label Encoding: The categorical labels ('Human', 'AI-generated-chatGPT', 'AI-generated-Gemini') have been Encoded using one-hot Encoding. [3]

You can see text data preprocessing pipeline here:

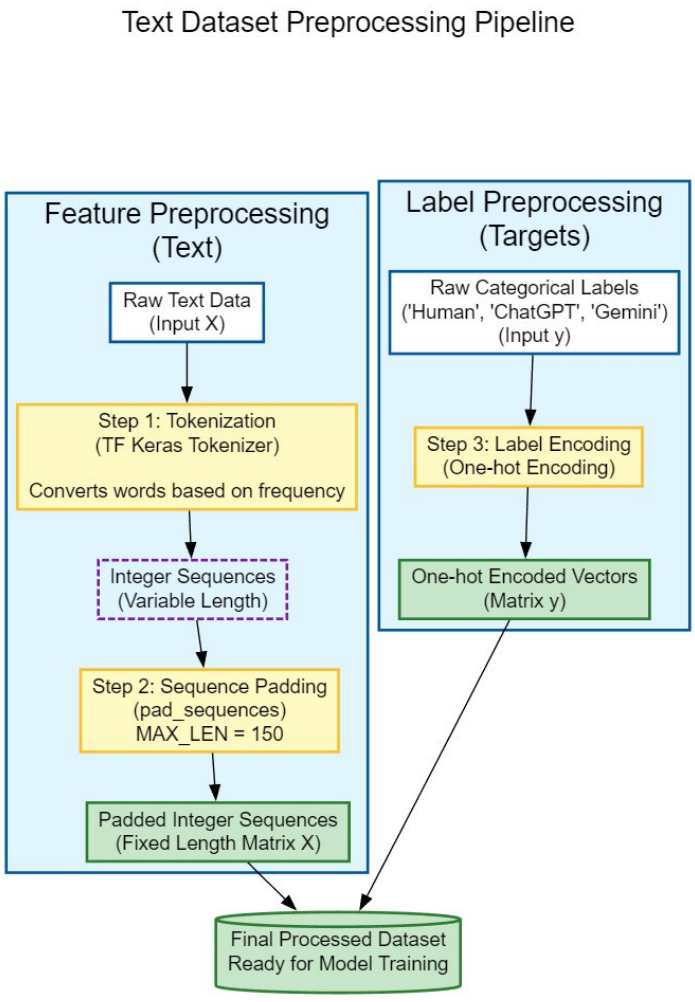

Figure2: Text dataset preprocessing pipeline

2.2.3 Audio Preprocessing Steps

Our model, a computer vision model namely MobileNetV2, cannot take audio data directly. We have designed a rigorous audio to image processing pipeline: [4]

1. Loading & Resampling: For audio files, librosa is used for the load function to resample the audio into 16,000 Hz. [10]
2. Duration Standardization: All audio files are cropped/padded to standardize them to exactly 4 seconds.Files shorter than 4 seconds are silently padded, and longer files are cut short. [10]
3. Mel-Spectrogram Extraction: We use 128 Mel bands to get the Mel-Spectrogram. This helps to effectively represent the frequency characteristics of the human voice. [10]
4. Log-Scale Conversion: In this step, we transform the power spectrogram into decibels to resemble the sensitivity of the human ear. This requires [10]
5. Normalization: The values of the spectrogram are normalized within a scale of 0-255, then converted into grayscale images. [10]
6. Vertical Flip: We reverse the images to correctly align the bottom portion with the lowest frequencies. [10]
7. Final Output: The output files are processed in the form of .png images. [10]

You can see images preprocessing pipeline here:

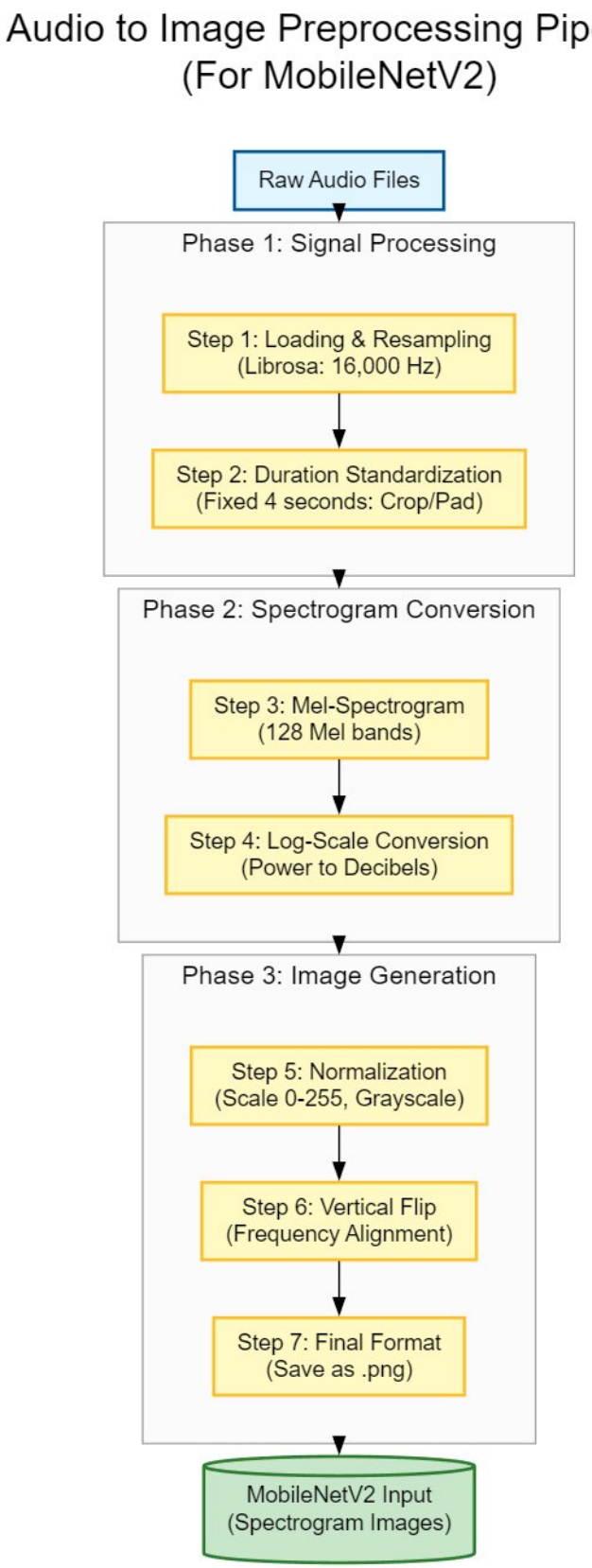

Figure 3: Audio dataset preprocessing pipeline

2.2.3 Image Preprocessing Steps

1. Each image is loaded from its file path within the dataset during data reading. [4]
2. Color Space Standardization: All images are converted to RGB format. This ensures that every image has three color channels (Red, Green, Blue), regardless of the original image format. [4]
3. Image Resizing: All images are resized to a fixed resolution of 224 × 224 pixels. This size is consistent with the input requirements of the pretrained CNN model. [4]
4. Conversion to Tensor: Images are converted into tensors so they can be processed by the deep learning model.Pixel values are scaled to the range [0, 1]. The tensor dimensions are arranged as (Channels, Height, Width). [5]
5. Image Normalization: Normalization is applied using ImageNet mean and standard deviation. This step aligns the input data distribution with the data used to pretrain the model, improving training stability and performance. [5]
6. Label Assignment: Each image is assigned a numerical label based on its class: [4]
   - Real image
   - AI-generated image
7. Batch Preparation: Preprocessed images and their labels are grouped into batches. During training, data is shuffled to reduce bias. During validation and testing, shuffling is disabled to preserve data order. [5]
8. Inference-Time Preprocessing: When a new image is provided for prediction:

   The same preprocessing steps are applied. The image is prepared in the correct format before being passed to the model. This ensures consistency between training and inference. [4]

   You can see images preprocessing pipeline here:

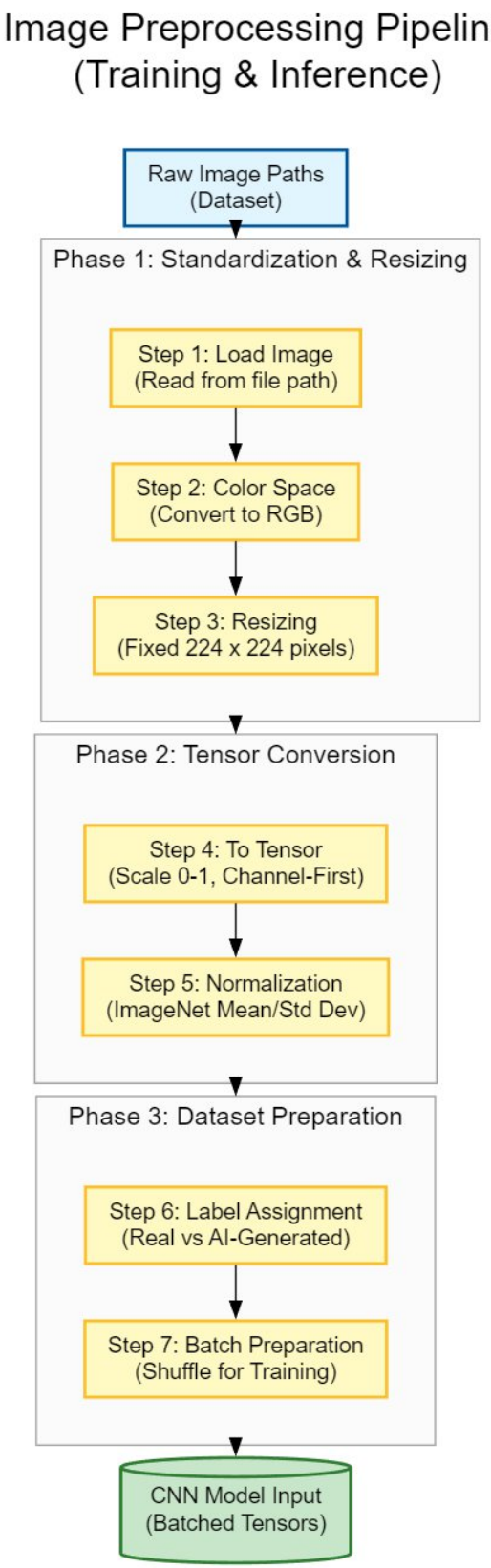

Figure 4: Images dataset preprocessing pipeline

CHAPTER 3. METHODOLOGY

3. METHODOLOGY

3.1 General Framework

The proposed solution for this task is a four-modality system. The deepfake detection model will focus on a video classification task, while the text detection using AI will concentrate on a Natural Language Processing task. Image model will focus on a photos classification task (Real or Fake) and finally audio model will classify voices if they are fake or not. [1], [2], [3]

The models will use a combination of deep learning techniques as follows:

3.2.1 Video Classification Methodology for Deepfake Detection

The method adopted in the deepfake video Detection is a sequence analysis instead of a classification analysis. The method starts with a preprocessing phase, in which the MTCNN is used to detect and crop faces in raw video frames in order to remove background noise. The frames are then formulated as sequences (for example, 20 frames) in order to preserve context. During feature extraction, a lightweight convolutional neural network named EfficientNet-B0 is utilized to extract spatial information in every individual frame. The spatial information is then submitted to a Long Short-Term Memory network, which undergoes temporal analysis to investigate discrepancies in sequences to identify peculiarities in blinking and lip patterns that might not be identified in a deepfake video. Finally, a fully connected layer is utilized to classify a frame as "Real" or "Fake."

Look at video model Framework:

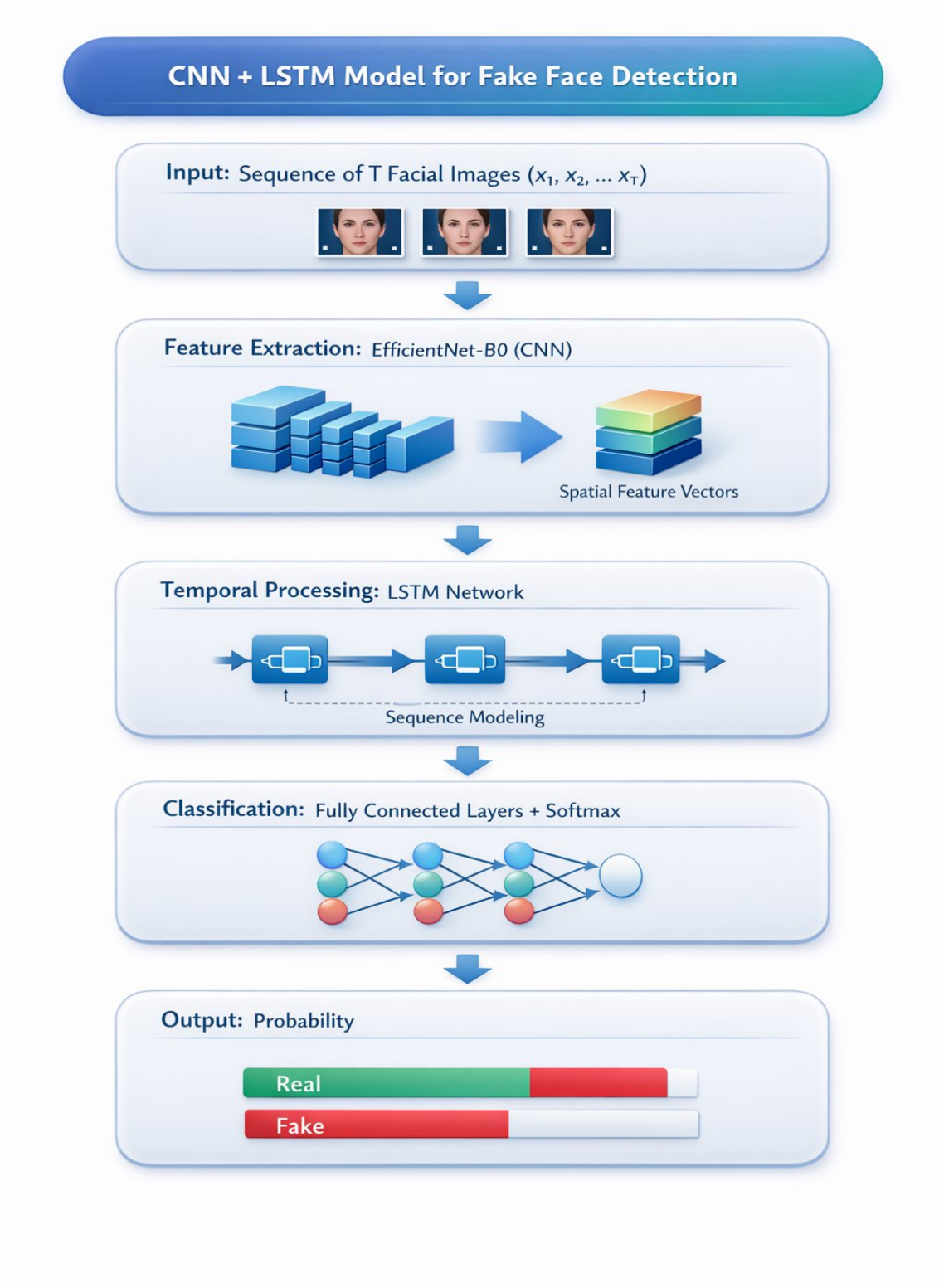

Figure 5: Video classification model (Deepfake Detection)

3.2.2 Text Classification Methodology (Gemini vs. ChatGPT vs. Human)

The method of text detection employs a hybrid deep learning model, which detects local patterns of style and distant semantic connections in the text. Preprocessing techniques used involve tokenizing and padding the sequences to have a fixed length of 150 tokens. The model incorporates 1D Convolutional layers (Conv1D) and a Bidirectional Long Short-Term Memory component (Bi-LSTM). Actually, in this model, the use of the Conv1D layer enables it to extract features by identifying recurring phrases or particular n-grams commonly linked with artificial intelligence model-generated texts. Meanwhile, the Bi-LSTMs allow it to read the texts in two ways, from the beginning and from the end, thus facilitating its ability to comprehend the entire context of the story. Hence, the entire model makes it easier to distinguish between the natural level of complexity in human-created texts and the semantically most likely and smoother pattern in texts generated from artificial intelligence models such as ChatGPT and Gemini.

Look at text model Framework:

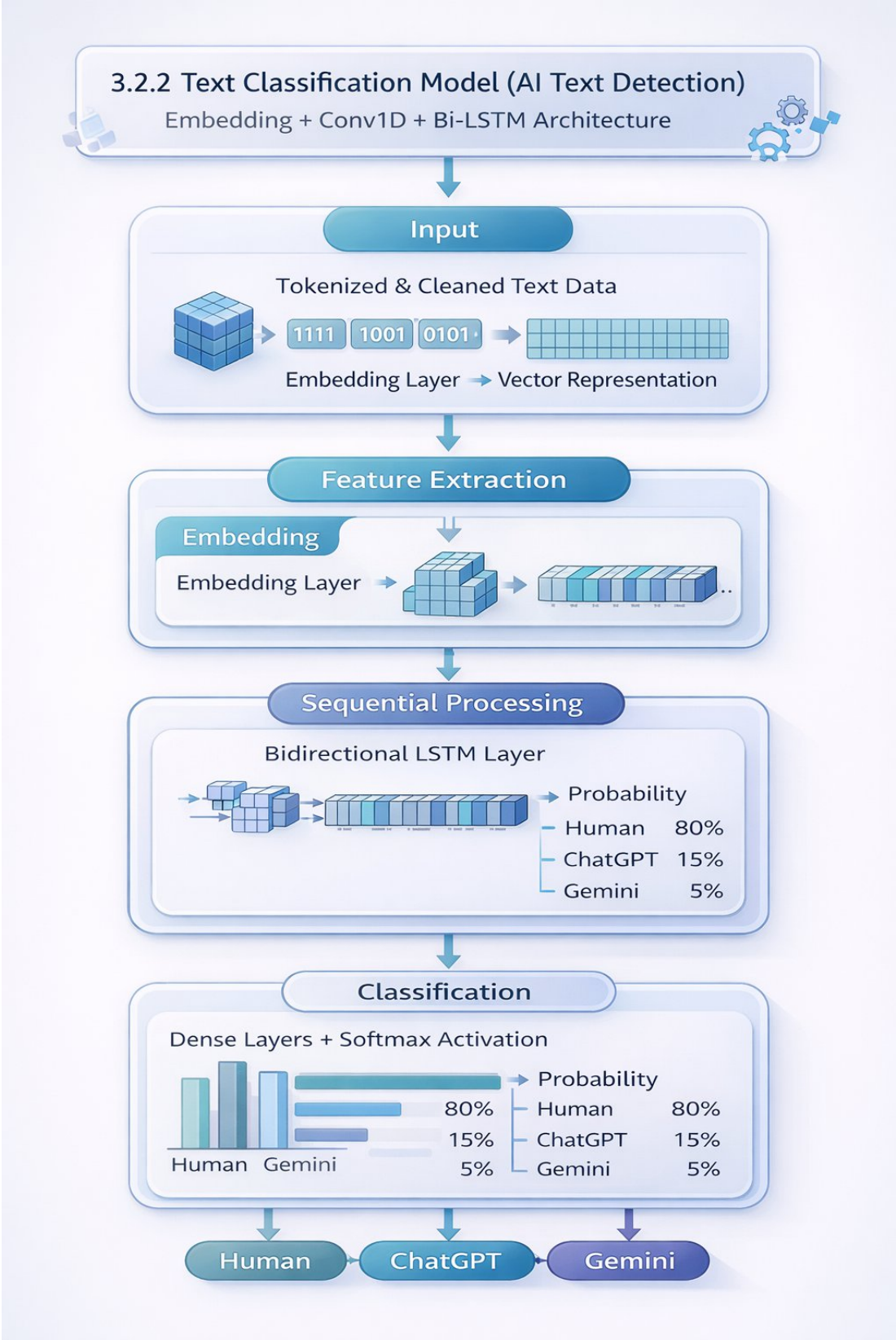

Figure 6: Text classification model (AI Text Detection)

3.2.3 Audio Classification Methodology (Synthetic Voice Detection)

Our method for identifying audio works well as transforming a signal processing task into a classification problem. In the Preprocessing step, we leverage the Librosa library, which converts the raw waveform data of the recorded audios into Mel-Spectrograms, which represent graphically the frequency content of the audios over time. The resulting Mel-Spectrograms can then be transformed to a logarithmic scale, as humans perceive sound more accurately on a logarithmic scale, before being finally written as images. For Classification, MobileNetV2, which is a lightweight CNN model, is employed. MobileNetV2 applies its ability to recognize images' visual properties to the Mel-Spectrograms' visual properties, recognizing artifacts such as the lack of smooth transitions or the presence of interruptions within frequency regions that are the hallmarks for text-to-speech conversion, voice conversion, or TTS services.

Look at audio model Framework:

Figure7: Audio classification model

3.2.4 Methodology for Image Classification (AI vs. Real Photos)

"The detection method aims at these minute, pixel-level details that creep in as a consequence of synthetic images being generated," says Wei Dang, a University of California, Los Angeles, computer science graduate student who participated in developing the system. "But before going through any further processing, we apply a uniform preprocessing to all inputs," says Dang. "We convert all images to RGB, resize all images to 224×224, and then further apply normalization based on ImageNet statistics." At the core of this system is ConvNeXt-Tiny, which is a modern CNN architecture that enhances the canonical ResNet blocks. "We train our model to detect these high-frequency relics and some oddities in the structure that GANs and diffusion-based images often forget to remove," says Dang. "The model will only give a binary probability as output, which tells you if an image is real or if it's a fake image generated by some artificial intelligence."

Look at images model Framework:

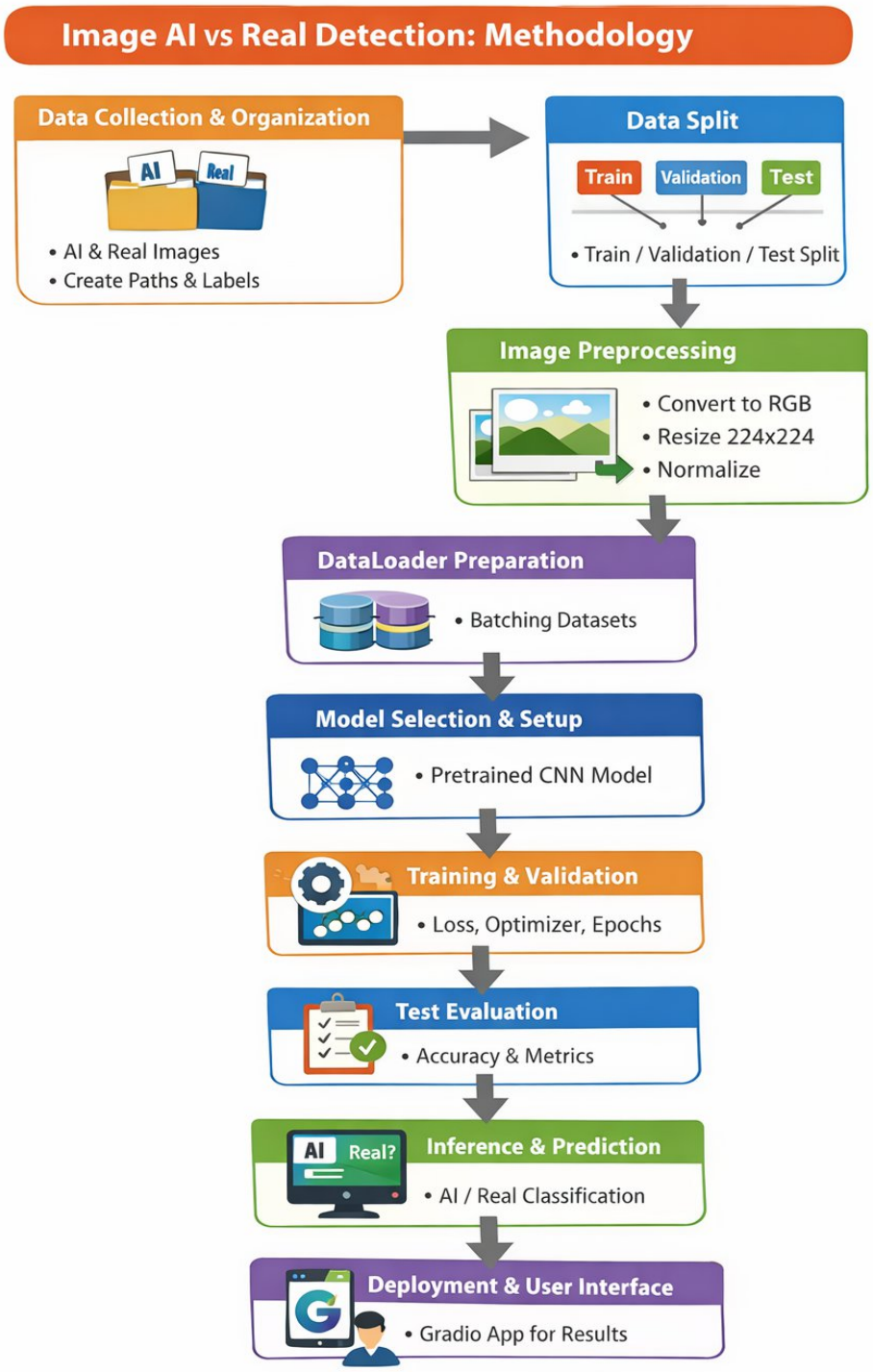

Figure 8: Images classification model

3.3 Background on The Used Algorithms

- EfficientNet (Video): A family of convolutional neural networks using a Compound Scaling Approach. In this work, EfficientNet-B0 is a light yet highly effective feature extractor that uses depth-wise separable convolutions to reduce computational cost. [5]
- Long Short-Term Memory (LSTM) / Bi-LSTM: LSTMs handle the "vanishing gradient problem" in conventional RNNs by using "gates" (input, output, and forget). For Video, this enables the model to "remember" facial structures across frames to identify distortions. While in Text, the Bidirectional variant analyzes the input in both modes. This results in twice as many context inputs for human writing style and probability generation. [8]
- 1D Convolution (Conv1D): Sliding window filter for text sequences. It proves highly useful for finding repetitive phrases or certain syntactic structures that are typical of language models. [5]
- MTCNN (Video Preprocessing) A multi-task cascaded approach applied for face detection and alignment tasks. This helps ensure that the video classifier always receives perfectly centered and cropped faces. [8]
- While in Audio, Convolutional Neural Networks (CNNs): These networks use filters to scan images for patterns. In our case, CNN looks for unnatural breaks or "smoothness" in the frequency bands of the spectrogram that are characteristic of AI-generated voice. Class Weighting Algorithm: To address the "failure" mentioned in Chapter 1, we utilized the "Balanced" weighting method from the Scikit-Learn library. This calculates a weight wj for each class j inversely proportional to class frequencies: (Wj = n samples / n classes * n samplesj) [6]

This will ensure that in the model, the rare data points "Fake" will be considered very important updates during training. [5]

CHAPTER 4. RESULTS AND DISCUSSION

4. RESULTS AND DISCUSSION

4.1 Experiments Setup

Given the massive amount of data we're handling, we need powerful hardware (such as GPUs), which we've been able to provide through the Colab and Kaggle platforms.

We created an identical experimental environment on the Kaggle platform based on NVIDIA Tesla T4 GPUs to ensure the reproducibility of our results and the strict evaluation of our architecture.[10]

4.1.1 Video Model Experiment Setup

Hardware Environment

We adopted a hybrid hardware solution, making use of a local machine for development, debugging, and data processing, while relying on a cloud computing infrastructure for training on large data sets due to memory constraints.

- Local Development Environment:
  - Device: Lenovo IdeaPad Gaming 3 (2021)
  - Processor: AMD Ryzen 7 5800H 8
  - Graphics: NVIDIA GeForce RTX 3060 (6 GB VRAM)
  - Memory: 24 GB RAM
- Usage: For initial coding prototyping, light model testing (MobileNetV2), and constructing a data pipe.
- Training Environment (Cloud):
  - Platform: Kaggle Kernels/ Google Colab
  - Accelerator: 2 x NVIDIA Tesla T4 Graphics Cards (Data Parallelism enabled)
  - VRAM: 15GB per GPU
  - Total effective video memory: 30 GB
  - Purpose: Used for training the final EfficientNet-B0 + LSTM model, which allowed the use of a larger batch size and mixed-precision training.

- Software Frameworks & Libraries

The project was written in Python version 3.10. The primary Deep Learning library for building the final models was PyTorch, which was selected due to the support for a dynamic computation graph and better memory management capabilities compared to the TensorFlow library.

- The main libraries and tools used in the process are:
  - Deep Learning: PyTorch (version 2.0+), torchvision, timm.
  - Computer Vision: OpenCV for the extraction of frames from the video, MTCNN for the face detection and alignment tasks, PIL or Python Imaging Library for handling images.
  - Data Processing: NumPy, Pandas, scikit-learn (splitting the data into train/test sets and setting weights)
  - Optimization: GradScaler in PyTorch for Mixed Precision Training (FP16) to better manage memory consumption during the processing of the sequence.
  - Dataset and Hyperparameters Configuration: The experiments were conducted on a Unified Video Dataset, which consisted of datasets from the Deepfake Detection Challenge (DFDC) as well as FaceForensics++.
  - Input Resolution: The input resolution of the images was changed to 224 × 224 pixels to satisfy the input size requirement for the EfficientNet-B0 model.
  - Sequence Length: This is set at a fixed value of 20 frames per video. This ensures that there is adequate temporal context without beingmemory-intensive.
  - Batch size: 16, taking advantage of the dual GPU setup to run the computation in parallel.
  - Optimizer: AdamW, which has a learning rate of 1e-4
    Loss Function: Binary Cross Entropy with Logits (BCEWithLogitsLoss)
  - Sampler: For oversampling the minority class “Real” and achieving aequable distribution of “Real” and “Fake” videos in each training batch, a WeightedRandomSampler was used.

4.1.2 Text Model Experiment Setup

Data Preparation and Preprocessing:

The model was developed and tested using the AI&Human_Content.csv data set. Data selection was rigorous with a focus on removing rows with missing values for either the text and label columns. Data selection was done such that only three classes are retained, namely 'Human-written', 'AI-generated-chatGPT', and 'AI-generated-Gemini'.

The text data were preprocessed for neural network processing in the following manner:

- Tokenization: A tokenizer with a maximum vocabulary size of 15,000 words was used to identify the most frequent words. Out-of-vocabulary words are replaced with a token named <oov>.
- Sequence Padding: The length of all text sequences was normalized to 150 tokens. If the length is less than the fixed length, the sequence is padded with zeros at the end (post-padding), and for lengths greater than the fixed length, the sequence is truncated at the end (post-truncation).
- Encoding: The class labels were integer encoded and then one-hot encoded using categorical encoding to satisfy the requirement of the multi-class classification problem.
- Data Splitting: The data after processing was split into a training data set (80%) and a validation data set (20%) by using stratified sampling for splitting.

Hardware and Environment:

- The experiments were performed on a local machine with high-end hardware that ensured fast processing for training the deep learning parts.
- GPU: NVIDIA GeForce RTX 3060 Laptop GPU.
- Compute Policy: To better manage memory during training and improve training speeds, the TensorFlow Mixed Precision Compute Policy (mixed_float16) strategy was used in the project. Mixed precision allows the model to use 16-bit math when appropriate while keeping variables at the stable 32-bit precision

Hyperparameters and The training procedure was subject to a specific set of hyperparameters, which were aimed at balancing the complexity of the models and their performance in terms of convergence.

The table below shows text model architecture and training configurations:

Table 1: Text Hyperparameters and configuration:

Parameter	Value	Description
Epochs	20	The number of complete passes through the training dataset.
Batch Size	64	The number of samples processed before updating the model weights.
Optimizer	Adam	Adaptive Moment Estimation for efficient gradient descent.
Loss Function	Categorical Cross entropy	Chosen for multi-class classification tasks.
Embedding Dimension	100	The size of the vector space for word representation.
LSTM Units	128	The number of memory units in the Bidirectional LSTM layer.
Dropout Rate	0.5	The probability of dropping units to prevent overfitting.
CNN Filters	128	Number of output filters in the convolution layer.
Kernel Size	5	The length of the 1D convolution window.

4.1.3 Audio Model Experiment Setup

- Environment: The model was trained on Kaggle using NVIDIA Tesla T4 GPUs.
- Hyperparameters:
  - Batch Size: 32
  - Epochs: 10
  - Optimizer: Adam (Learning Rate = 0.001)
  - Loss Function: Binary Cross entropy

- Weighting: Pre-calculated class weights were injected into the training loop model.fit(..., class_weight=weights).

4.1.4 Image Model Experiment setup

- Hardware Setup: The experiment for the image classification tasks was set up in a cloud-based training setup with the intention of enabling effective computation and reproduction of results. It was designed to automatically use GPU computation whenever possible.
- Training Environment (Cloud): Kaggle Notebooks
- Accelerator: 2 x NVIDIA Tesla T4 GPUs (Data parallelism enabled where supported by the execution environment) VRAM:15 GB per GPU

It allowed for sufficient computation power for efficient training of deep convolution neural networks for images.

Software Frameworks & Libraries: The implementation of the images detection model was conducted using the Python programming language. The principal framework to be utilized for deep learning was PyTorch.

Some of the key libraries used include:

- Deep Learning: PyTorch, torchvision
- Pretrained Models: ConvNeXt-Tiny (ImageNet)
- Image Processing: PIL (Python Imaging Library)
- Data Handling: NumPy, Pandas
- Deployment & Testing: Gradio (for Interactive Image Prediction)

Dataset and Preprocessing Configuration

Data Splitting:

- The data was split between a training set comprising 80% of the data and a validation set comprising 20%

* A constant random seed has been utilized for the reproducibility of the split.

Image Preprocessing:

- All images were resized to a dimension of 224x224 pixels in order to be in line with the input size for the ConvNeXt architecture.

- The images were converted to tensors and normalized with ImageNet mean and standard deviation.

The table below shows ConvNeXt architecture and training configurations:

Table 2: Image Model configuration and hyperparameters

Category	Parameter	Description
Model Architecture	Backbone	ConvNeXt-Tiny pretrained on ImageNet
	Classifier Head	Modified final classification layer for binary classification (Real vs Fake)
Training Configuration	Batch Size	32
	Number of Epochs	15 epochs with Early Stopping
	Optimizer	AdamW
	Learning Rate	1e-4
	Loss Function	Binary classification loss (e.g., BCEWithLogitsLoss or Cross-Entropy, depending on implementation)

4.2 Evaluation Metrics

We evaluated the model using:

1. Accuracy: The percentage of total correct predictions.

2. Loss: The binary cross-entropy loss, indicating how confident the model is in its correct predictions.
3. Confusion Matrix: To visualize False Positives vs. False Negatives

4.3 Experiments

Our experimental method was iterative. We aimed to find the best setup for detecting deepfakes in real-world situations. We started with simple spatial analysis and gradually moved to more complex temporal modeling. We improved our data pipeline and model setup at each stage based on performance results and hardware limits.

4.3.1 Video Model Experiments

We carried out three distinct phases of experimentation to identify the best model architecture.

Phase 1: Frame-Based Classification (Baseline Attempt)

- Architecture: Xception Network (Image-only)
- Configuration: Single-frame input, pre-trained on ImageNet.
- Methodology: In our first experiment, we treated video deepfake detection as an image classification problem. We used the Xception architecture to classify individual frames taken from videos as either "Real" or "Fake," without considering the sequence of frames.
- Results & Analysis:
- Training Performance: The model achieved a high validation accuracy of about 87%, indicating good feature extraction abilities.
- Deployment Failure: Despite strong training metrics, the model struggled during real-time testing on unseen videos. It had a high false positive rate and often misclassified real videos as fake.
- Conclusion: This experiment showed that spatial analysis alone is insufficient. The model was overfitting to image flaws like blur, noise, or compression quality instead of detecting the manipulation itself. It lacked the temporal awareness needed to identify inconsistencies in motion, blinking, or facial expressions over time.

Phase 2: High-Complexity Temporal Modeling (Resource Constraints)

- Architecture: Xception + LSTM
- Configuration: Sequence length of 20 frames, fine-tuning enabled.
- Methodology: To address the lack of temporal context, we extended the Xception model by adding an LSTM (Long Short-Term Memory) layer. This changed the model from an image classifier to a video sequence analyzer. We tried to fine-tune the entire network by unfreezing the CNN backbone to learn specific deepfake features.
- Results & Analysis:
- Technical Challenges: This architecture became too costly for our hardware. The Xception backbone, with around 22 million parameters, combined with the memory needed to store gradients for a sequence of 20 frames, led to frequent out-of-memory (OOM) errors on the GPU.
- Conclusion: While the architecture seemed sensible in theory, it was unworkable in practice. We decided a lighter, more efficient backbone was needed to balance depth with memory limits.

Phase 3: EfficientNet-LSTM with Balanced Sampling (Final Success)

- Architecture: EfficientNet-B0 + LSTM.
- Configuration: Sequence length of 20 frames, WeightedRandomSampler, Mixed Precision Training (GradScaler).
- Methodology: In the final iteration, we replaced the Xception backbone with EfficientNet-B0, a model designed for efficiency and speed without losing accuracy.

We also made key improvements to the data pipeline:

- Imbalance Handling: We found a significant class imbalance in our combined dataset (4,132 Fake vs. 1,089 Real videos). We used a WeightedRandomSampler to ensure each training batch had a balanced representation of both classes.
- Mixed Precision: We applied PyTorch's GradScaler and autocast to reduce memory usage, allowing us to maintain a sequence length of 20 frames without crashing.
- Data Augmentation: We combined the "DeepFake Detection" (DFD) and "FaceForensics++" datasets to improve generalization.

- Results & Analysis: This setup proved to be the most reliable. The WeightedRandomSampler corrected the model's bias toward the majority class, and the LSTM effectively captured temporal inconsistencies. The model showed high confidence and accuracy on unseen validation data, correctly distinguishing between real and manipulated footage where earlier models had failed.

4.3.2 Text Model Experiments

In this manner, the capability of the proposed Bi-LSTM model in distinguishing between text written by a human and text produced using AI tools (ChatGPT and Gemini) can be explored via several experiments.

A. Initial Implementation and Challenges ("The Bad Try")

The first approach used in conducting the experiments was directly uploading the raw data (AI&Human_Content.csv) into the preprocessing phase. The performance was substandard with a model that generalized poorly and often classified different AI inputs as human content.

Upon analyzing the structure of the dataset, it was determined that there was a serious issue with the formatting of the data:

- Fragmentation: The original CSV source file didn’t have a whole document per row. The original essays were broken down by newline characters. For instance, a document would be broken down into several rows: title on row 1, body on row 2, subhead on row 3.
- Sparsity of the Target Label in the Data: In the data, only the first line of a document had the targeted label (for example, "AI generated--ChatGPT"), and the rest of the rows for the same documents were empty.
- Context Loss: Being trained on sentences and sentence fragments as opposed to well-formed, coherent text made it difficult for it to recognize structural and stylistic relationships found in AI-written text (transition sentences and paragraphs).

B. Data Reconstruction and Optimization (The Solution)

To address the problem of data fragmentation, a strong data cleaning and reconstruction process was incorporated before tokenization using the Pandas library. The remedial measures include:

- Label Propagation: A forward-fill (ffill) operation was used to propagate the document-level class labels from the initial row to all subsequent fragmented rows.
- Grouping: Unique group identifiers were created for each instance of a document to differentiate between distinct texts that have identical labels. - Reconstruction: The rows were grouped based on these identifiers, and the text pieces were merged using the newline character. With this methodological shift, the dataset was converted from being a set of disparate sentences into a corpus of full sentences. Thus, the Bi-LSTM became able to examine complete text contexts.

C. Experimental Setup

The final model was trained on the restored dataset with the following specifications:

- Architecture: Bidirectional Long Short-Term Memory network with 128 units, accompanied by an Embedding layer with 100 dimensions.
- Input Constraints: Maximum vocabulary size of 15,000 words. Input sequences are padded or truncated to a fixed size of 150.
- Training Procedure: 10 epochs, batch size = 64.

D. Results and Analysis

After optimization, the model performed very well in distinguishing among the three classes: Human, ChatGPT, and Gemini.

Qualitative analysis was carried out on unseen examples to establish how well-posed and robust to generalization the learned models are. The results showed high confidence values on the correct class labels, establishing that the models were able to learn distinct stylistic patterns per class.

This table can summarize the results of deployment text model:

Table 3: Prediction confidence on unseen test text data

Input Source	Content Topic	Predicted Label	Confidence Score	Result
CNN Article	Human migration & conflict	Human-written	99.97%	Correct
Gemini	Human trafficking	AI-generated-Gemini	99.78%	Correct

	explanation			
ChatGPT	CRISPR gene editing ethics	AI-generated-ChatGPT	99.92%	Correct

The experiments confirm that data reconstruction is indeed the key element in propelling model performance. By giving the Bi-LSTM model access to full document structures, rather than being presented with disconnected fragments, improved detection between natural variations in human-written data and patterns often found in AI-written data became possible.

4.3.3 Audio Model Experiments

We conducted training over 10 epochs.

- Epoch 1: The model started lower in terms of accuracy - around 38% while adjusting to weights.
- Epoch 2-5: After which it began to learn very fast. The accuracy jumped to ~95%, showing that the transfer learning was working.
- Epoch 10: The model converged.

4.3.4 Image Model Experiments

Binary Cross Entropy Loss (BCELoss) Saving a Model: * The model that performed the best has been stored for future use based on validation results. The goal of the image model experiments was to tackle the effectiveness of deep CNN models for discriminating AI-created images from authentic images. The experimental setup focused on training for stability, generalization, and robustness. The model showed fast convergence in the early training epochs, as there was a significant reduction in training loss. This shows that the pre-trained ConvNeXt-Tiny model backbone was able to extract good features from the images.

During this training process, it is evident that improvement took place in both training and validation metrics. The highest validation accuracy and lowest validation loss were attained at Epoch 14 with a validation accuracy of 99.95% and a validation loss of 0.0015. Additionally, at this epoch, training accuracy reached 100% and training loss touched a negligible value of 0.0004.

To reduce the issue of overfitting and to ensure that the best network is selected, the use of the "early stopping method" helped in halting the training process. This occurred when, for two consecutive epochs, the validation loss did not improve.

This final chosen image model has been able to attain a Best Validation Accuracy Score of 99.95% to demonstrate the efficiency of the ConvNeXt-Tiny model along with the process of transfer learning as well as appropriate image preprocessing. This all together indicates that the proposed image model is reliable for real-world applications involving detection in AI-generated images.

4.4 Results Analysis Description

This section provides a detailed look at the performance metrics from the training and validation phases of the video (Deepfake), text (AI vs. Human), Audio and image detection models. The results show that all models achieved high accuracy and were able to generalize well. They effectively reduced overfitting through their design choices and callback methods.

4.4.1 Video Results Analysis Description

The Deepfake Video Detection model used an EfficientNet-B0 backbone along with an LSTM layer. It was trained for a maximum of 15 epochs with an Early Stopping mechanism that had a patience of 3 epochs.

- Training Progression: The model learned features quickly. In the first epoch, it reached a validation accuracy of 87.44%, which was much better than the initial training accuracy of 73.70%. This difference indicates that the data augmentation methods, like Color Jitter, helped strengthen the model, making the non-augmented validation set easier to classify at first.
- Peak Performance: The model achieved its best performance at Epoch 5, with a Validation Accuracy of 93.87% and a Validation Loss of 0.1774. This shows a high ability to differentiate between real and altered video sequences.
- Stability and Overfitting Control: Training accuracy steadily increased, reaching 96.92% by Epoch 8. There was a brief dip at Epoch 6, where validation accuracy fell to 78.71%. This likely happened due to random variations during batch optimization. However, the model bounced back by Epoch 7, reaching 93.42%.

- Early Stopping: The training process ended automatically after Epoch 8 because the validation accuracy did not surpass the 93.87% mark from Epoch 5 for three straight epochs. This automatic stop was essential to prevent the model from fitting too closely to the training noise. It ensured that the final saved model maintained the highest possible generalization capability.
- Conclusion: The EfficientNet+LSTM architecture was very effective, achieving about 93.9% accuracy on new video data. This validates its ability to capture both spatial details (through CNN) and temporal inconsistencies (through LSTM) found in Deepfake media.

Here you can see the video model performance in the training:

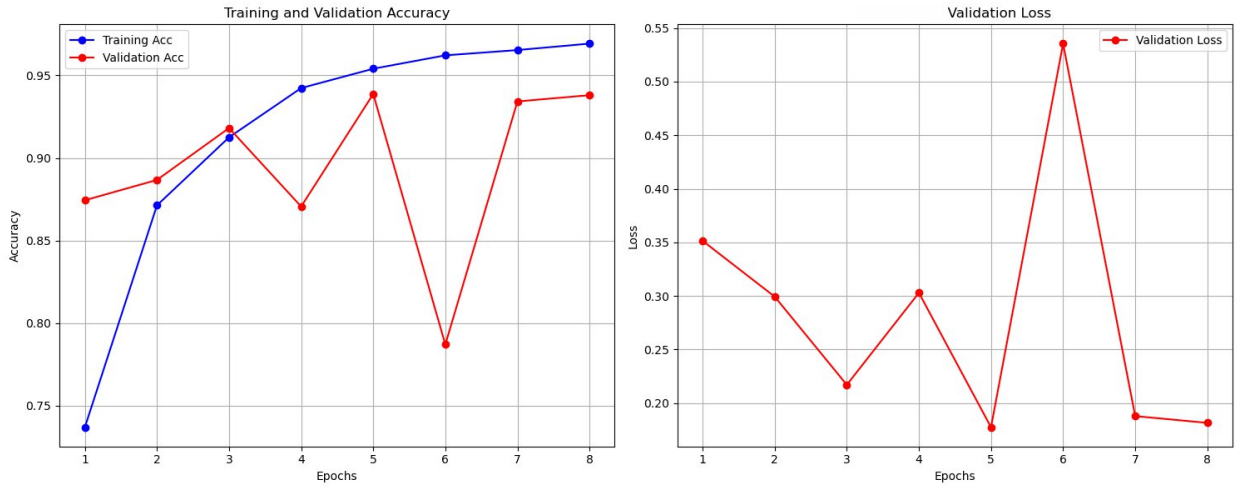

Figure 9: Deepfake video detection model performance during training

4.4.2 Text Results Analysis Description

- The Text Detection model used Hybrid CNN-Bi-LSTM architecture. It was trained on a dataset with three classes: Human-written, AI-generated (ChatGPT), and AI-generated (Gemini). Convergence and Accuracy: The model showed very fast convergence. By Epoch 2, the validation accuracy reached 97.55%. The training accuracy reached 100% from Epoch 10 onward, showing that the model had fully learned the patterns in the training dataset.
- Final Metrics: After 20 epochs, the model recorded a final Validation Accuracy of 98.94% and a low Validation Loss of 0.0435. The similarity between the validation and training accuracies suggests that the model generalizes well and is not experiencing severe overfitting, even with high training scores.

- Class Distinction Capabilities: Testing on unseen data samples showed that the model could finely distinguish between different Large Language Models (LLMs), rather than just between Human and AI outputs.

  Here is sample of the performance (on unseen data):

  - Gemini Detection: The model accurately identified Gemini-generated text with confidence of 98.58%.
  - ChatGPT Detection: It recognized ChatGPT-generated text on the same topic with confidence of 99.92%.
  - Human Detection: It classified human-written journalistic text with complete confidence.

- Conclusion: The Hybrid CNN-Bi-LSTM model showed nearly perfect performance in this classification task. The high confidence scores, usually above 98%, during the prediction phase confirm that the extracted features, likely a mix of local n-gram patterns from CNN and long-term dependencies from LSTM, are strong indicators of AI-generated content.

Here you can see the Text model performance in the training:

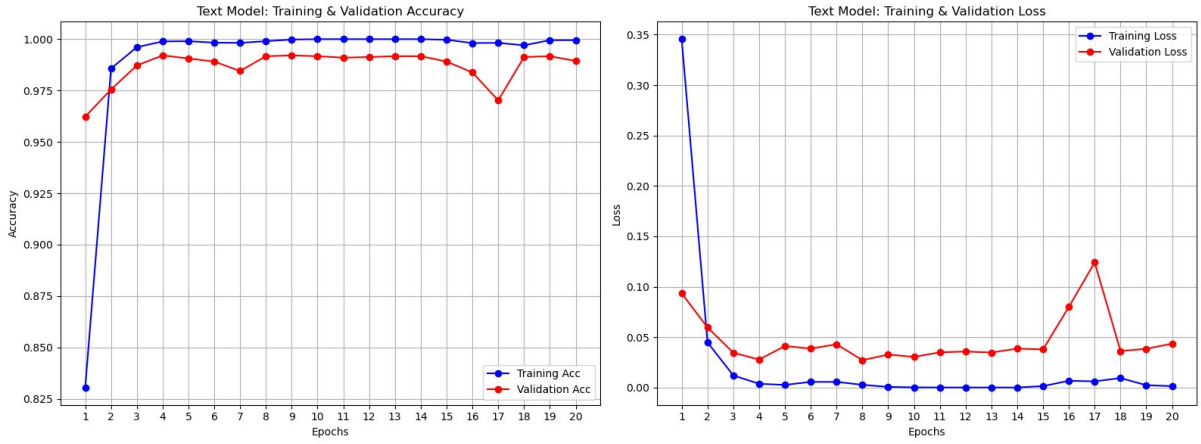

Figure 10: Text detection model performance during training

4.4.3 Audio Results Analysis Description

The validation accuracy of the final model was around 97.7%.

- Impact of Class Balancing: Contrary to our previous unsuccessful attempts wherein the model cared less for the fake class due to imbalanced classes, our final model cared greatly

for the "Fake" class. The graph indicated steady reduction in loss value with minimized loss value at a small validation loss value of 0.0602.

Here you can see the audio model performance in the training:

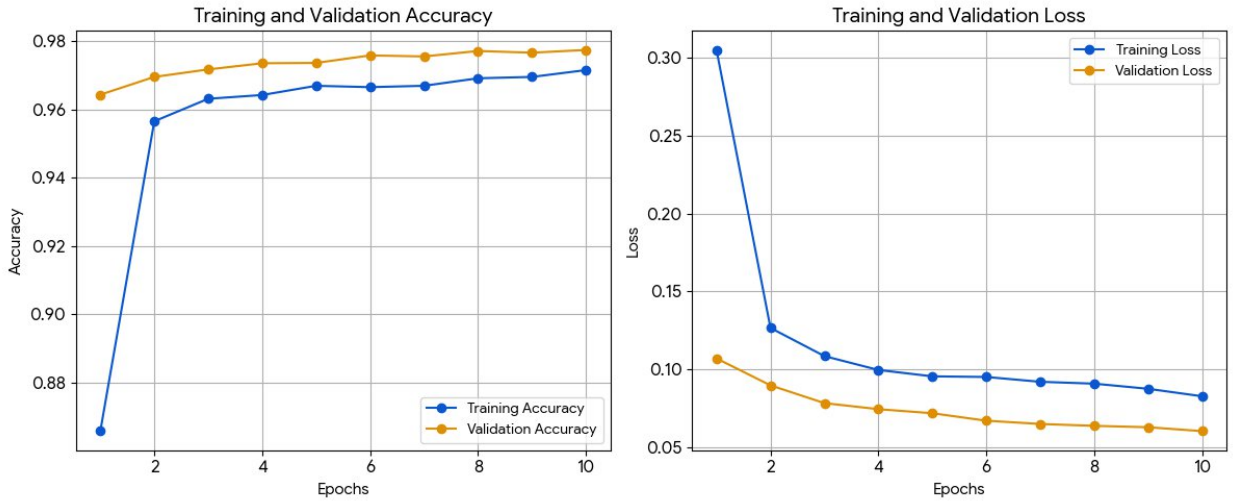

Figure 11: Audio detection model performance during training

4.4.4 Image Results Analysis Description

The model reached a nearly perfect accuracy of 99.95%. Although it was supposed to run for 15 rounds, it stopped early at round 14 because it had already achieved its best performance and was no longer improving. Overall, the results show that the model is extremely successful and highly accurate. Here you can see the images model performance in the training:

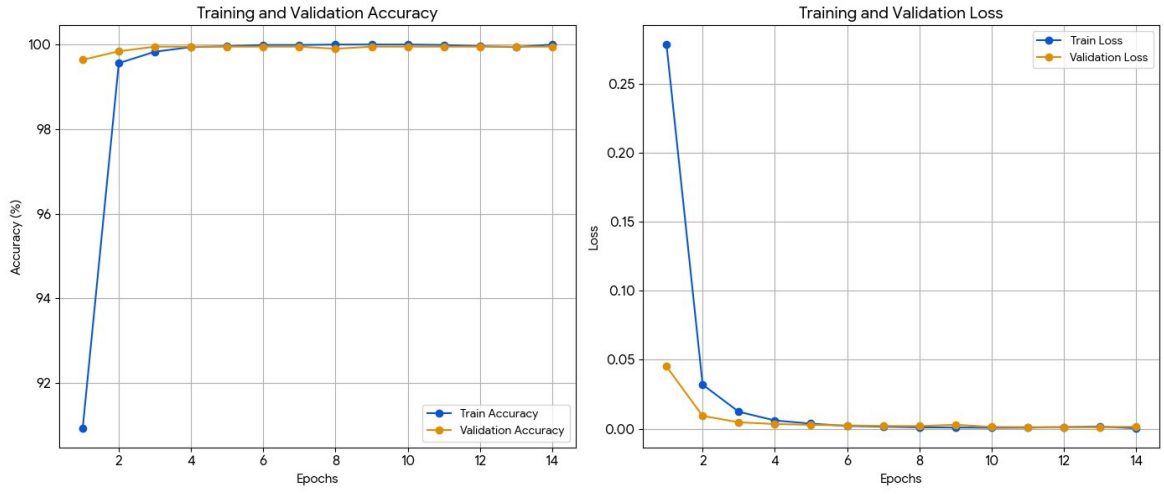

Figure 12: Image detection model performance during training

CHAPTER 5. INTERFACE DESIGN

5. INTERFACE DESIGN

5.1 Overview

This phase of interface design serves as a bridge between very complex models of deep learning (CNN-LSTM models, EfficientNet, TensorFlow text models). Essentially, "the primary focus of a Deepfake Detection System's interface would be to provide a simple and accessible environment where users can easily check the authenticity of digital media without having to understand much about how things work." [4], [5], [7], [8]

The system is implemented as a Web Application using the Flask framework. This ensures cross-platform compatibility so that it is possible to access the application using any standard web browser on desktop or mobile phones. The system design focuses on simplicity and feedback so that the result obtained (Real or Fake) is displayed clearly without ambiguity.

5.2 User Interface Architecture

The application uses the popular architectural design Model-View-Controller (MVC), modified to suit the online environment:

- View (Templates): The view templates are implemented using HTML5 and CSS3. They also employ a templating engine called Jinja2. These elements form part of the presentation layer. They are responsible for user input, either a file or plain text, and output from a detection system.
- Controller (Flask Routes): The Python server code (app.py) acts as the controller. It is responsible for routing the received media uploads to the respective preprocessing tasks and triggering the respective AI tasks.
- Model (AI Inference): The backend uses the pre-trained models' weight files (best_deepfake_model.pth, best_audio_model.keras, etc).

5.3 Navigation and Site Map

The application is organized around four distinct detection modules that are accessible via a navigation hub. This website design is deliberate in keeping different types of media separate to avoid user confusion.

- Home/Landing Page: It is the core page which contains all the tools' links.
- Video Check Module: This is the interface used for uploads in both MP4/AVI video formats
- Audio Check Module: Interface for analyzing WAV and/or MP3 audio files.
- Text Check Module: Input Form designed for analysis of automatically generated text.
- Image Check Module: interface for analyzing images and return classification result.
- Page/HTML Check Module: It is a comprehensive module for analyzing content on a webpage, which can include image, text, audio or video content.

5.4 Detailed Interface Description

5.4.1 Home Page (Dashboard)

The Home Page is where browsers first arrive, and its purpose is to be both informative and welcoming.

- Header: Shows the project branding and the navigation bar for easy access.
- Cards: The main area has specific cards or buttons called "Video Detection," "Audio Detection," "Text Analysis," and "Page Analysis." Design Rationale: "The interface uses large clickable areas to make the design touch-friendly."

This figure represents Home Page (Dashboard)

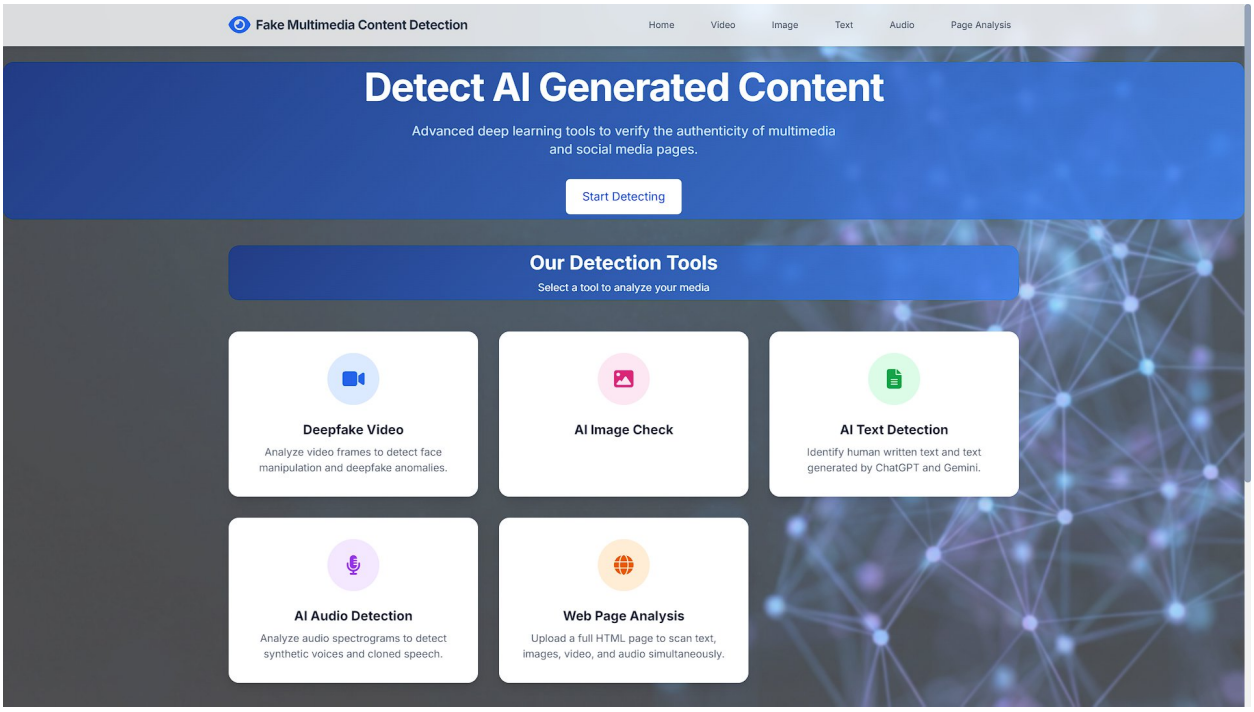

Figure 13: Home page (Dashboard)

5.4.2 Video Detection Interface

It is the most critical module of the project and uses the EfficientNet-LSTM architecture.

- Input Mechanism: Files can be uploaded by using a drag-and-drop area or by using a typical 'Browse' button. Supported formats for video upload are .mp4, .avi, or .mov.
- Processing Indicator: When the "Analyze" button is triggered, the GUI shows a processing indicator, such as a spinner, as the backend analyzes the 20 initial frames of the video stream.
- Result Display:
- Visual Result/Verdict: The color-coded badge is clearly visible. While Green represents "Real Video," Red represents "Deepfake Detected."
- Confidence Score: The model's confidence is measured by a confidence score expressed in the form of a percentage value such as "98.5% Confidence."

- Logic: The sequence of 20 frames is extracted, processed, and an average probability result is obtained using the CNN-LSTM architecture.

Here we can see the upload page for video:

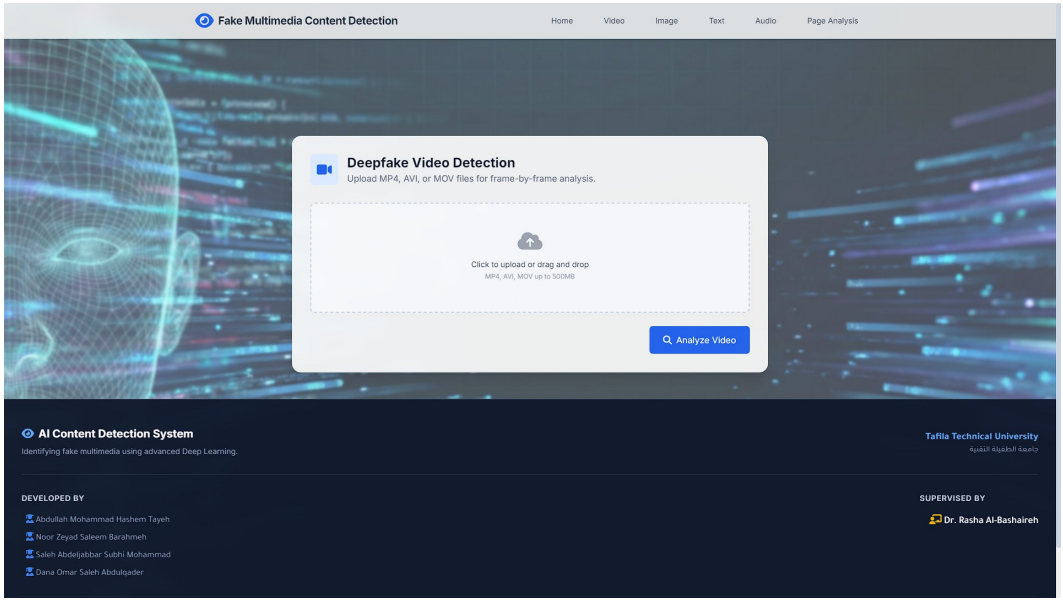

Figure 14: Video upload page.

Look at real result how looks:

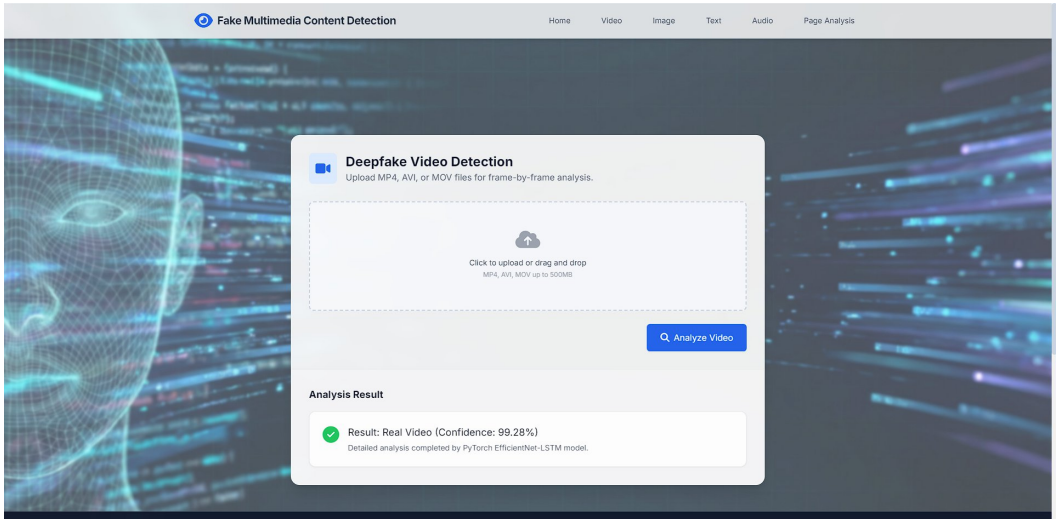

Figure 15: Real video result (Green Label).

You can see the fake result here:

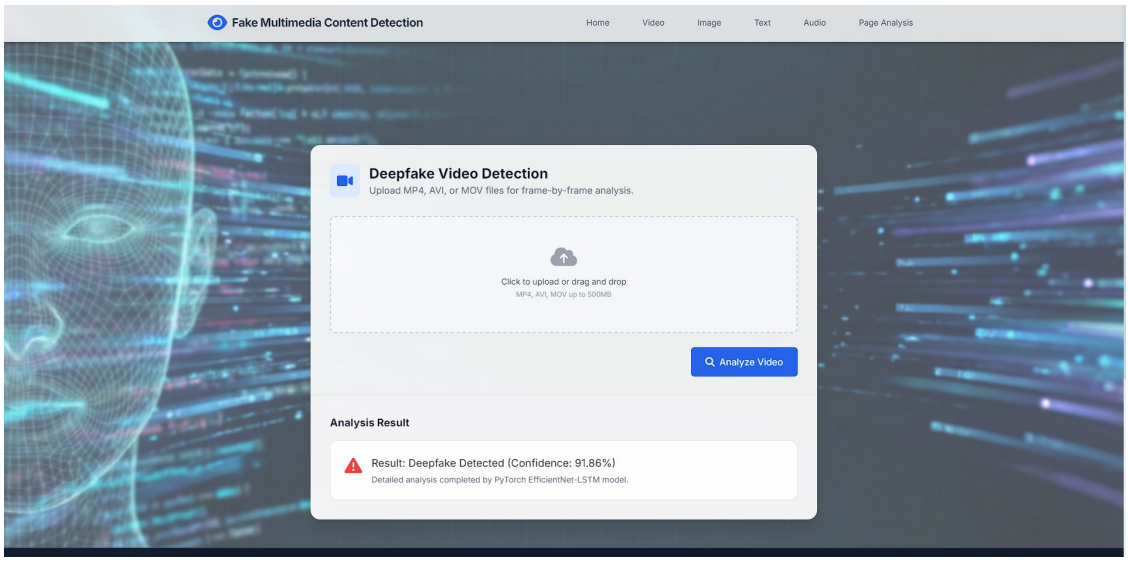

Figure 16: Deepfake video result (Red Label).

5.4.3 Audio Detection Interface

- Input Mechanism: The input mechanism supports the uploading of audio file types in .wav, .mp3 and many other formats.

  Here is figure 17:

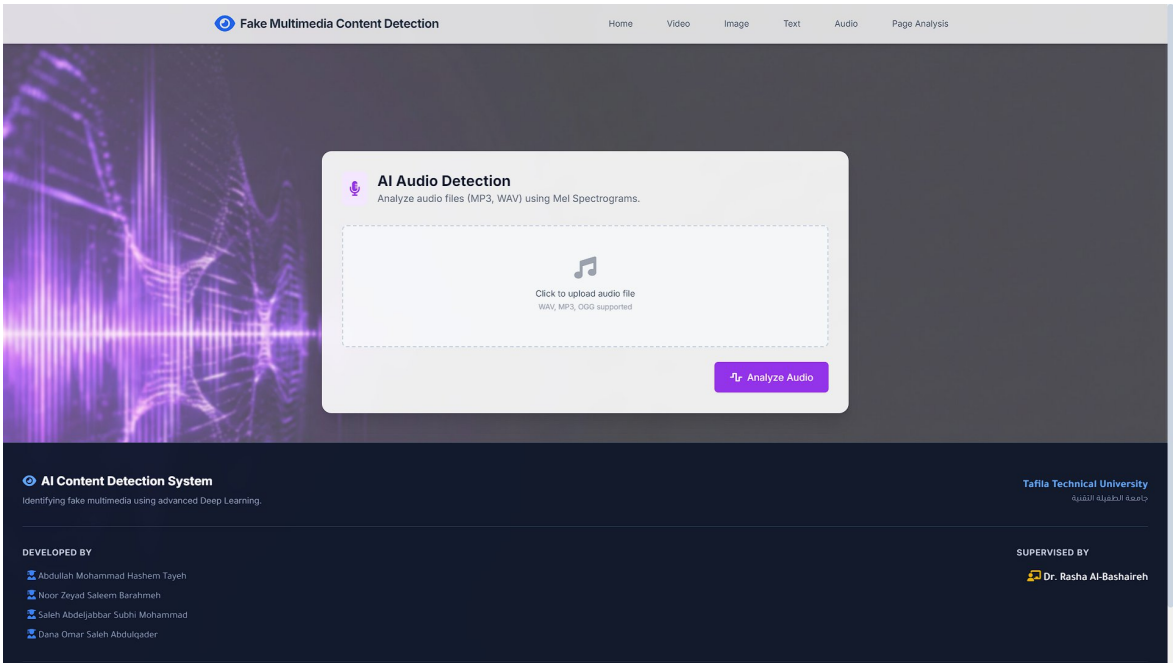

Figure 17: Audio upload page

- Processing: The backend converts the uploaded audio into Mel-spectrogram representations.
- Result Display: This is fully aligned with the video module and provides an interface that separates "Human Speech" (Real) and "AI/Spoofed Audio" (Fake) based on a prediction that comes from a TensorFlow model.

As described into these figures (18 & 19):

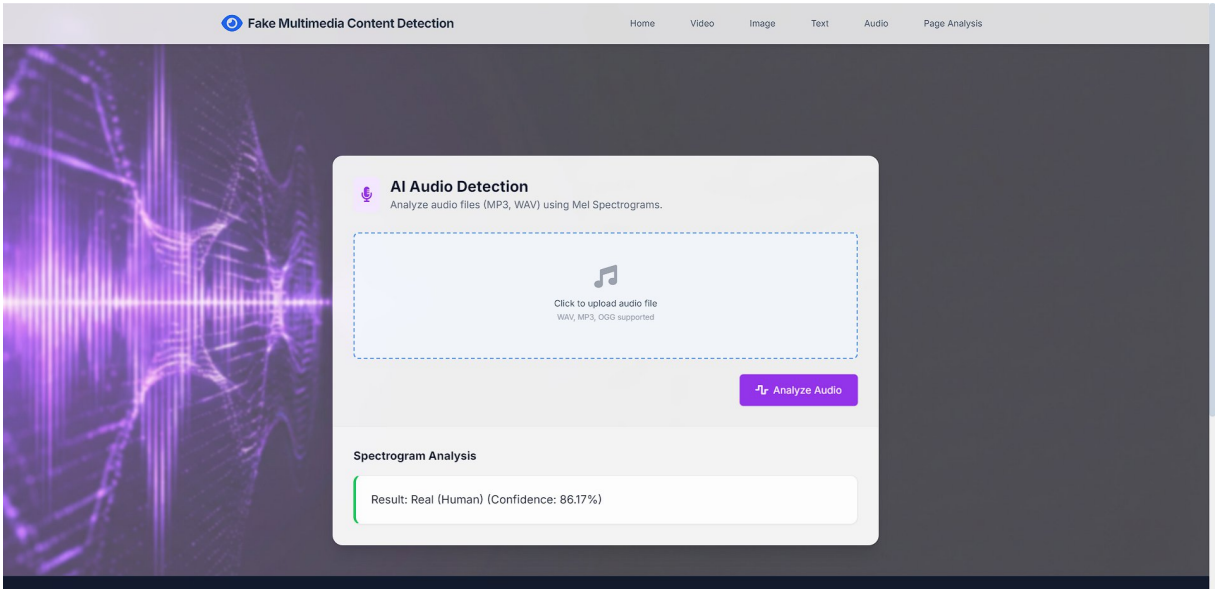

Figure 18: Real audio result

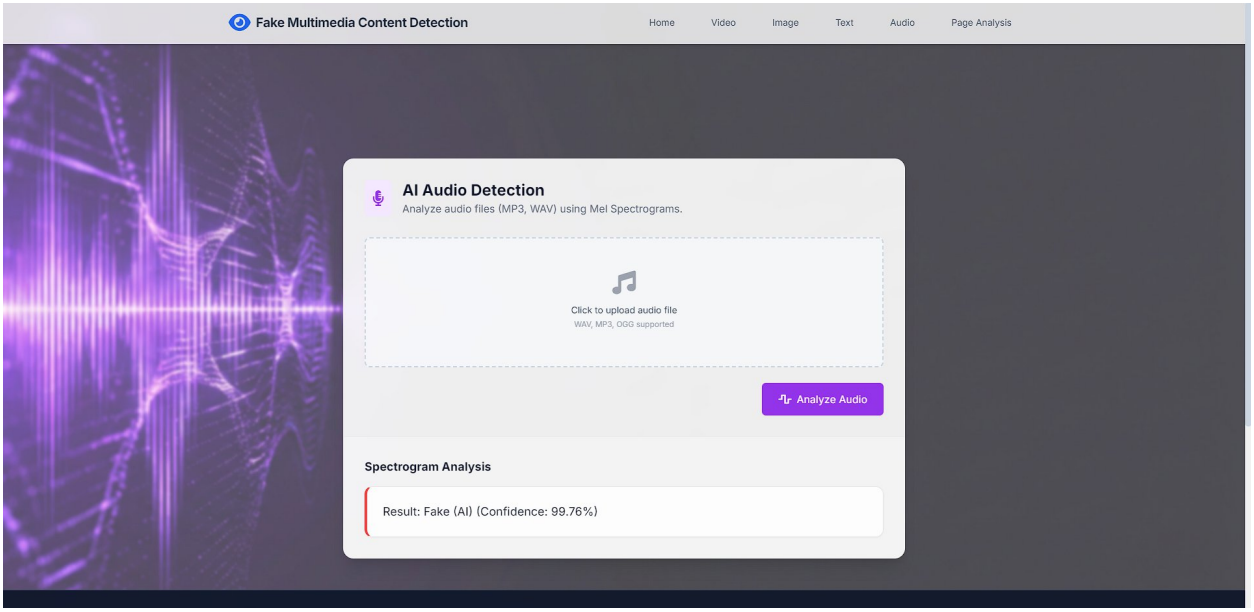

Figure 19: Fake audio result

5.4.4 Text Analysis Interface

- Input Mechanism: The text area provided allows users to paste news articles, essays, or Facebook posts.

  Look at figure 20:

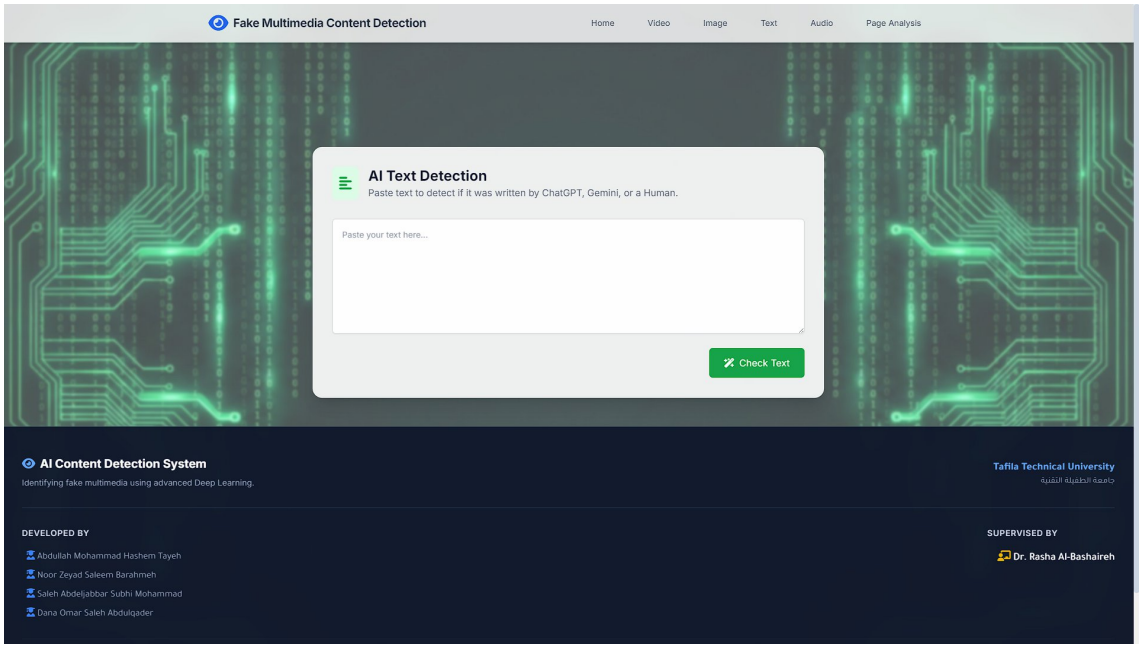

Figure 20: Text upload page

- Result Display: The result is presented in a more granular form, with the classification of the source of the text into distinct categories:
  - Human-written
  - AI-generated (ChatGPT)
  - AI-generated (Gemini)
  - Breakdown of Probability: This is followed by a detailed section that presents the breakdown of the probability distribution with respect to the three categories above.

As you can see below:

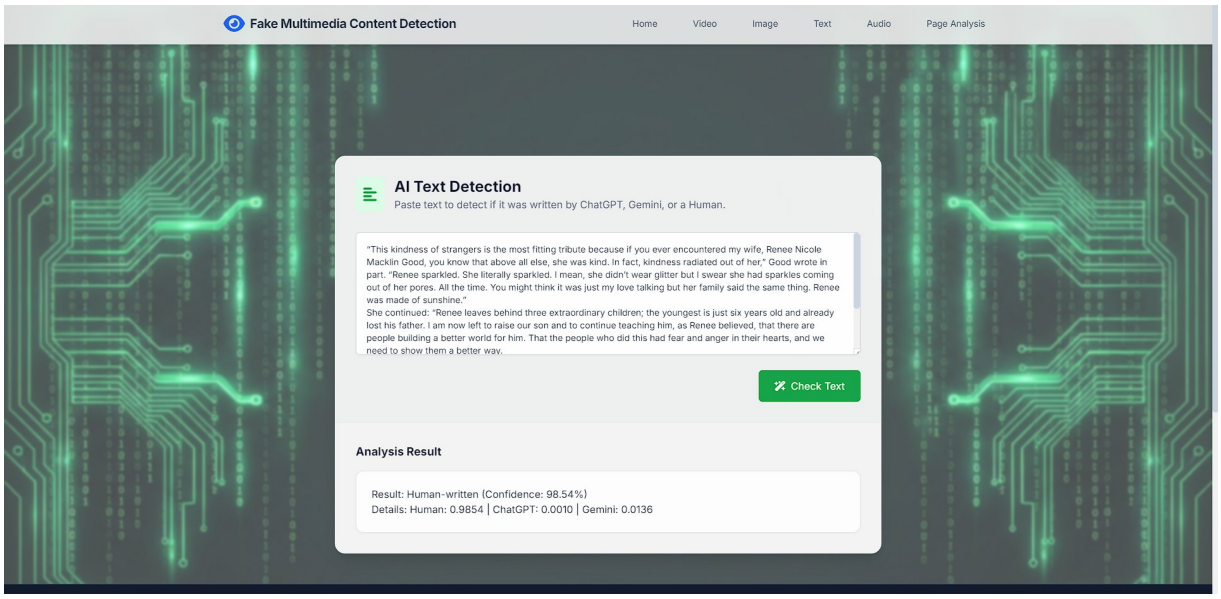

Figure 21: Text human-written result

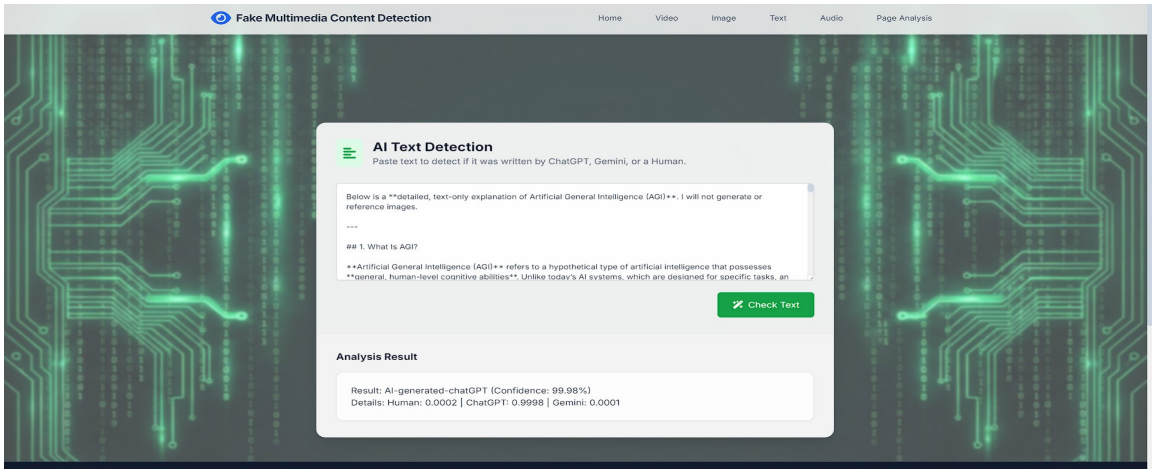

Figure 22: AI generated –chatGPT result

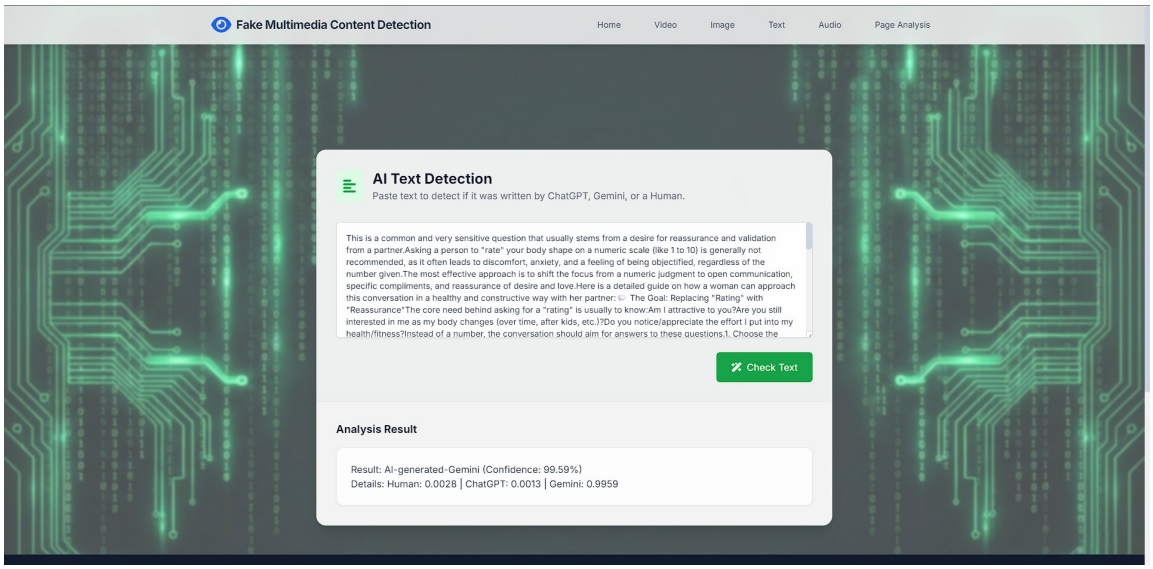

Figure 23: AI generated -gemini result

5.4.5 Images classification interface

- Input Mechanism: Image includes diverse environments like forests, cities, beaches, deserts, and more. Fake and Real images

  Look at figure 24:

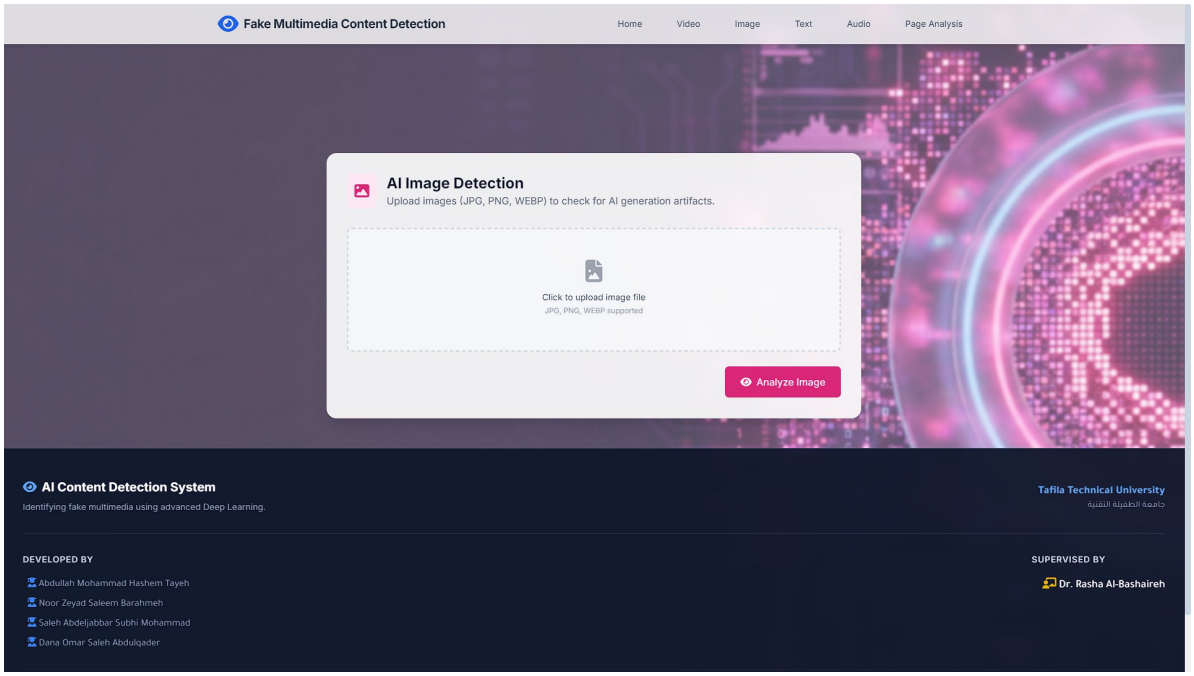

Figure 24: Image upload page

- Result Display: The result is presented in a more granular form, with the classification of the source of the text into distinct categories:
  - Real Images
  - Fake Images

  As you can see in these figures:

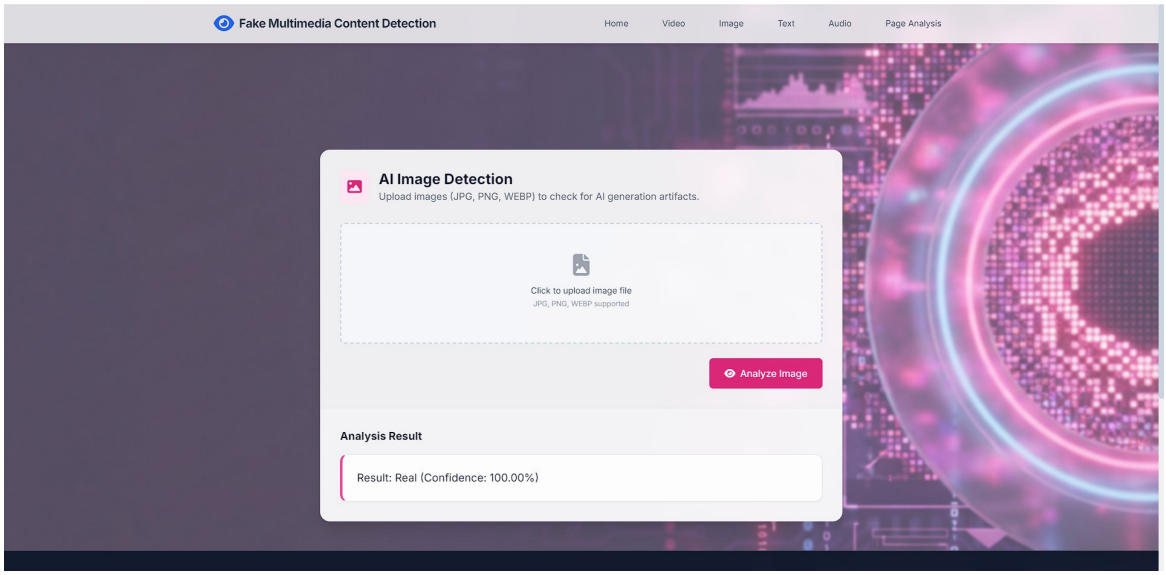

Figure 25: Real image result

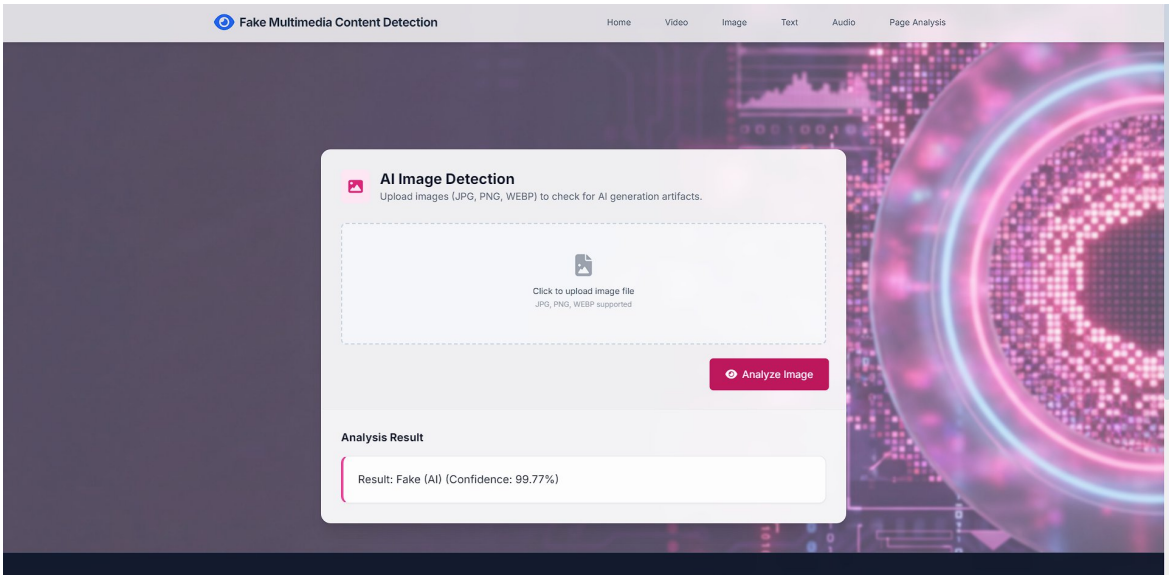

Figure 26: Fake image result

5.4.6 Page Analysis Interface

This sophisticated interface supports the uploading of an HTML file that may represent a social media site or a web page.

- Layout: The results are presented in a structured list format as "Posts."
- Content Breakdown: For every post discovered inside the HTML, the interface breaks down the content into Text, Image, Video, and Audio.
- Integrated Results: Each part is processed with its respective model results, which are then aggregated. For instance, a single post may show:

```
AI DETECTION RESULT
Text: Result: Human-written (Confidence: 100.00%)
Details: Human: 1.0000 | ChatGPT: 0.0000 | Gemini: 0.0000
Audio: Result: Real (Human) (Confidence: 99.67%)
```

As you can see in these figures:

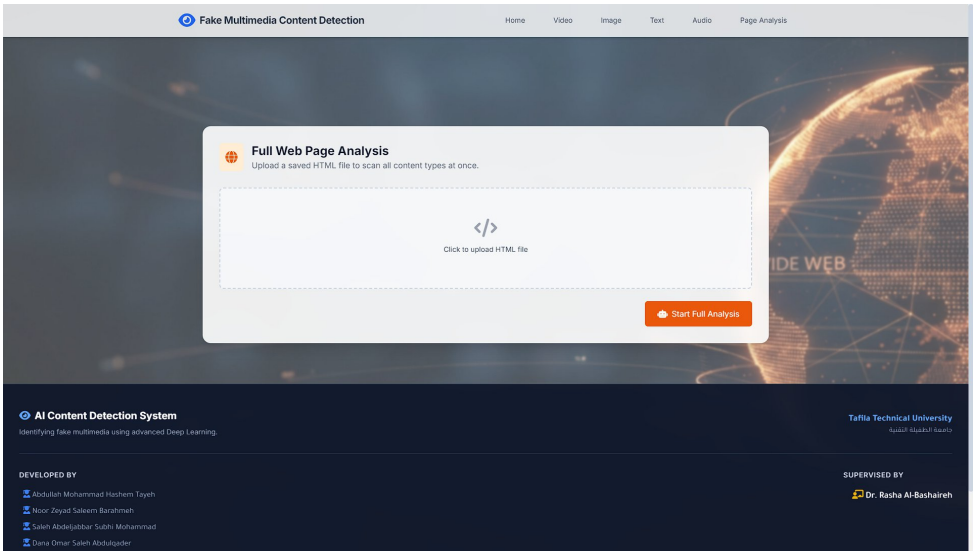

Figure 27: Page Analysis upload page.

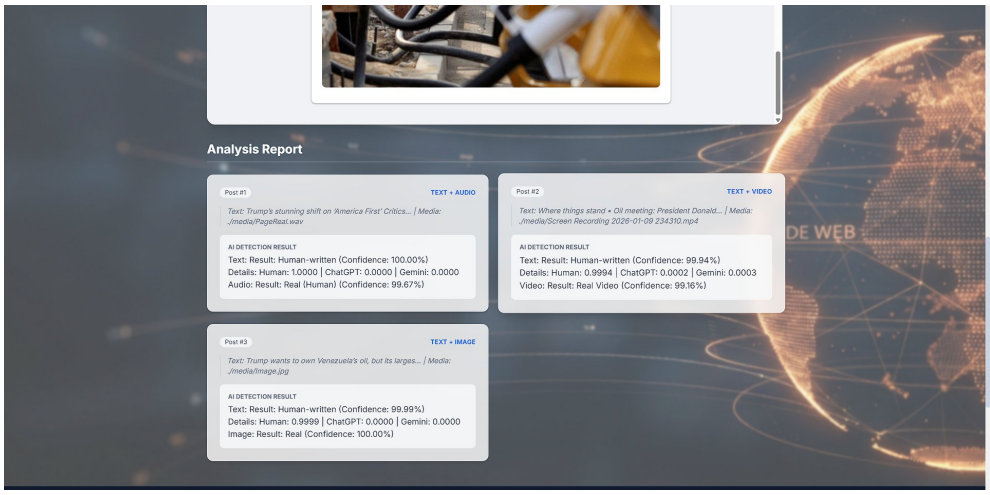

Figure 28: Result showing multiple media types analyzed at once (Real content).

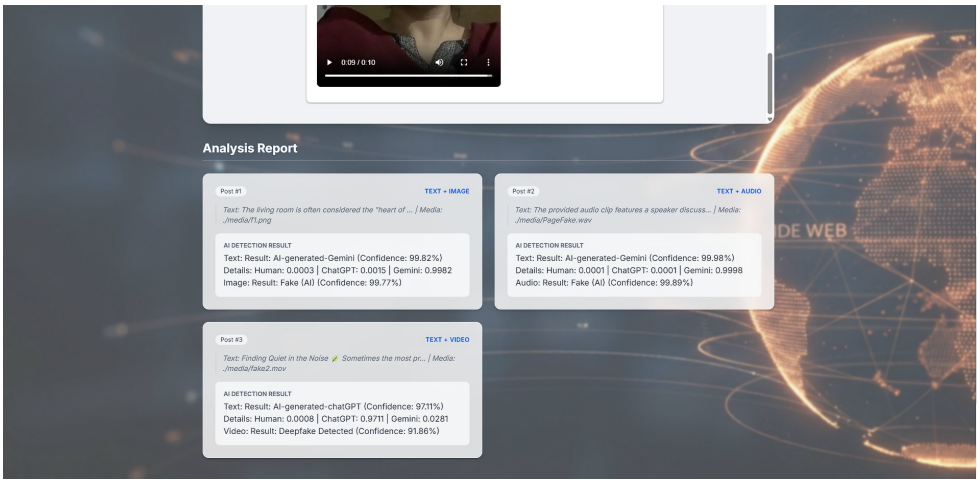

Figure 29: result showing multiple media types analyzed at once (Fake content).

5.5 User Experience (UX) Design Flow

The user journey has been made simplistic and pardonable. It works as follows:

- Input: Choose a module to input required information.
- Validation: Client-side validation takes place, checking for factors such as file extension and size, while server-side validation also happens for validation of file type, size, and so on. In case any of these checks fail (considering, for example, a .exe file being uploaded into a video detector), there will be an error message appearing at the top of the screen.
- Processing: The server computes a secure name for the file, temporarily stores the file, and

performs the inference.

- Output: The prediction will appear on the results page.
- Cleanup: The file that is uploaded for privacy reasons and space efficiency is processed and deleted while results come out. It is designed to allow continuous testing without wasting the patient energy of the person who is operating.

5.6 Visual Design & Feedback

- Color Palette:
- Primary: Professional Blue/Slate to represent trust and security.
- Success: Emerald Green to highlight Authenticity & Results.
- Danger: Crimson Red to flag or deepfakes manipulation. Fonts and Colors: Sans-serif fonts such as “Roboto” or “Open Sans” are chosen for improved readability. Responsiveness: The page utilizes CSS FlexBox as well as CSS Grid, which makes all the elements scale well on any device.

CONCLUSION AND FUTURE RECOMMENDATIONS

This project represents a full-fledged four modality detection solution that will help combat the growing threat of "Deepfakes" and other forms of synthetic media that are currently caused by the increasing threats posed by "AI". The project adopts the strategy of "fighting AI with AI".

Empirical evidence verifies the effectiveness of the chosen deep learning models:

1. Video Detection: The combination of spatial feature extraction using EfficientNet-B0 and temporal analysis using an LSTM layer helped Video Detection achieve an accuracy of 93.87% on its validation set, showcasing its capability to detect discrepancies in motion and expression.

2. Analysis of Text: The combination of Hybrid CNN-Bi-LSTM yielded an accuracy level of 98.94%, able to successfully distinguish between human-written text and text produced by sophisticated large language models like ChatGPT and Gemini.

3. Audio Detection: Using MobileNetV2 in the analysis of Mel-Spectrograms treated the audio detection task as an image classification problem and reached convergence with very high accuracy (approximately 97.7%) in the detection of synthetic voices.

4. Image Detection: The ConvNeXt-Tiny model proved its robustness by achieving a maximum validation accuracy of 99.95% for distinguishing real photographs from artificial ones.

Moreover, a web application using Flask has been developed to make such complex models usable for non-technologists, providing them with "Real vs. Fake" decisions along with confidence values through an advanced interface. Thus, the project proves that despite the ever-advancing AI tools for creation, multimodal deep learning still acts as an indispensable tool for digital integrity.

FUTURE RECOMMENDATIONS

In spite of the robust performance exhibited by the suggested system, there are various promising areas that can be explored to increase the capabilities of the system. These areas include the amalgamation of the detection of fake followers and bots using artificial intelligence techniques. Social media users behavior, including the patterns of follower accumulation, the ratio of engagement, the frequency of posts, as well as interaction patterns, can be used to classify accounts that are spreading artificially increased fake/media content by using artificial intelligence.

Another potential area of improvement may be the use of techniques from the realm of graphs and social network analysis in an attempt to identify the coordinated spread of disinformation campaigns. By combining the analysis of the authenticity of the content with the analysis of the patterns at the network level, one may obtain an even more comprehensive understanding of the patterns of the spread of misinformation on the internet. A potential area of interest for the next phase of research is the use of transformer architecture in text and multimodal fusion models in an attempt to combine the analysis of the text with the audio and/or video in an attempt to improve the detection of the misinformation.

Lastly, incorporating it into an extensible browser extension or an API-based service for social media sites can significantly expand its practical applications. Such implementations will further emphasize its role as an “intelligent defensive shield against misinformation, deepfakes, or artificial social influences in the dynamic digital environment.”

LIST OF TABLES

LIST OF FIGURES

REFERENCES

[1] A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Nießner, "FaceForensics++: Learning to Detect Manipulated Facial Images," 2019 IEEE/CVF International Conference on Computer Vision (ICCV), pp. 1–11, 2019.

[2] N. Dufour, A. Gully, P. Karlsson, A. Vorbyov, T. Leung, J. Childs, and C. Bregler, "DeepFakes Detection Dataset by Google & Jigsaw," arXiv preprint arXiv:1909.11573, 2019.

[3] K. M. Hermann, T. Kocisky, E. Grefenstette, L. Espeholt, W. Kay, M. Suleyman, and P. Blunsom, "Teaching Machines to Read and Comprehend," Advances in Neural Information Processing Systems (NeurIPS), vol. 28, pp. 1693–1701, 2015.

[4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4510–4520, 2018.

[5] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," Proceedings of the 36th International Conference on Machine Learning (ICML), vol. 97, pp. 6105–6114, 2019.

[6] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks," IEEE Signal Processing Letters, vol. 23, no. 10, pp. 1499–1503, 2016.

[7] F. Chollet, "Xception: Deep Learning with Depthwise Separable Convolutions," 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1800–1807, 2017.

[8] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[9] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," Proceedings of the International Conference on Learning Representations (ICLR), 2019.

[10] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, and O. Nieto, "librosa: Audio and Music Signal Analysis in Python," Proceedings of the 14th Python in Science Conference (SCIPY), pp. 18–24, 2015.